

计算智能与启发式算法

硕士生课程讲义

第二部分：演化计算

第一节：最优化问题

最优化问题

- 最优化问题的求解模型如下公式所示

$$\text{Max } F(X), X \in D$$

- 其中 **D** 是问题的解空间， **X** 是 **D** 中的一个合法解。一般可将 **X** 表示为 **$X = (x_1, x_2, \dots, x_n)$** ，表示一组决策变量
- 最优化问题就是在解空间中寻找一个合法的解 **X** （一组最佳的决策变量），使得 **X** 对应的函数映射值 **$F(X)$** 最大（最小）

最优化问题

- 一般可描述为下述数学规划模型：

$$\begin{cases} \max F(X) = (f_1(X), \dots, f_n(X)) \\ \quad st \\ \quad \quad g_i(x) < 0, i = 1, \dots, m \\ \quad \quad h_j(x) = 0, j = 1, \dots, k \\ X \in D \end{cases}$$

当 $n = 1$ ，时为单目标优化

当 $n > 1$ ，时为多目标优化

当 $m = k = 0$ 时为无约束优化，否则为约束优化

X 取值离散为离散优化，若 $X \in D \subset \mathbb{R}$ 为连续优化

最优化问题的分类

- 根据决策变量 x_i 的取值类型，可以将最优化问题分为函数优化问题和组合优化问题两大类



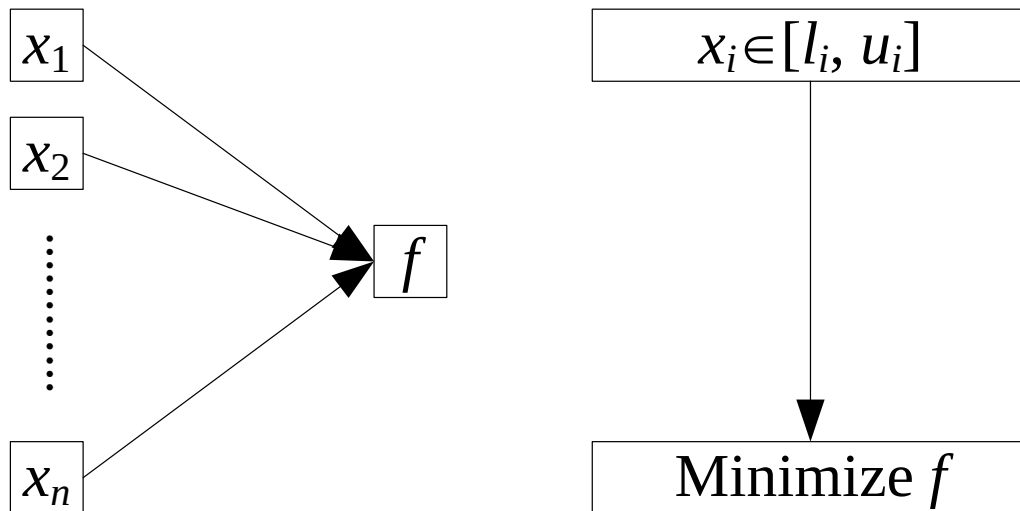
函数优化问题

决策变量均为连续
变量的最优化问题

组合优化问题

决策变量均为离散
取值的最优化问题

函数优化问题



- 例如：

$$f(x) = \sum_{i=1}^n x_i^2$$

其中， $n=30$ 表示问题空间的维数， $x_i \in [-100, 100]$ 是定义域，这个函数的最小值为0

- 这是一个最简单的函数优化问题

函数优化问题

- 很多科学实验参数配置和工农业生产实践都需要面临这种类型的最优化问题
- 在这种问题中，需要优化的变量的取值是某个连续区间上的值，是一个实数。各个决策变量之间可能是独立的，也可能是相互关联、相互制约的，它们的取值组合构成了问题的一个解
- 由于决策变量是连续值，因此对每个变量进行枚举是不可能的。在这种情况下，必须借助最优化方法对问题进行求解

例：无约束单目标，多峰

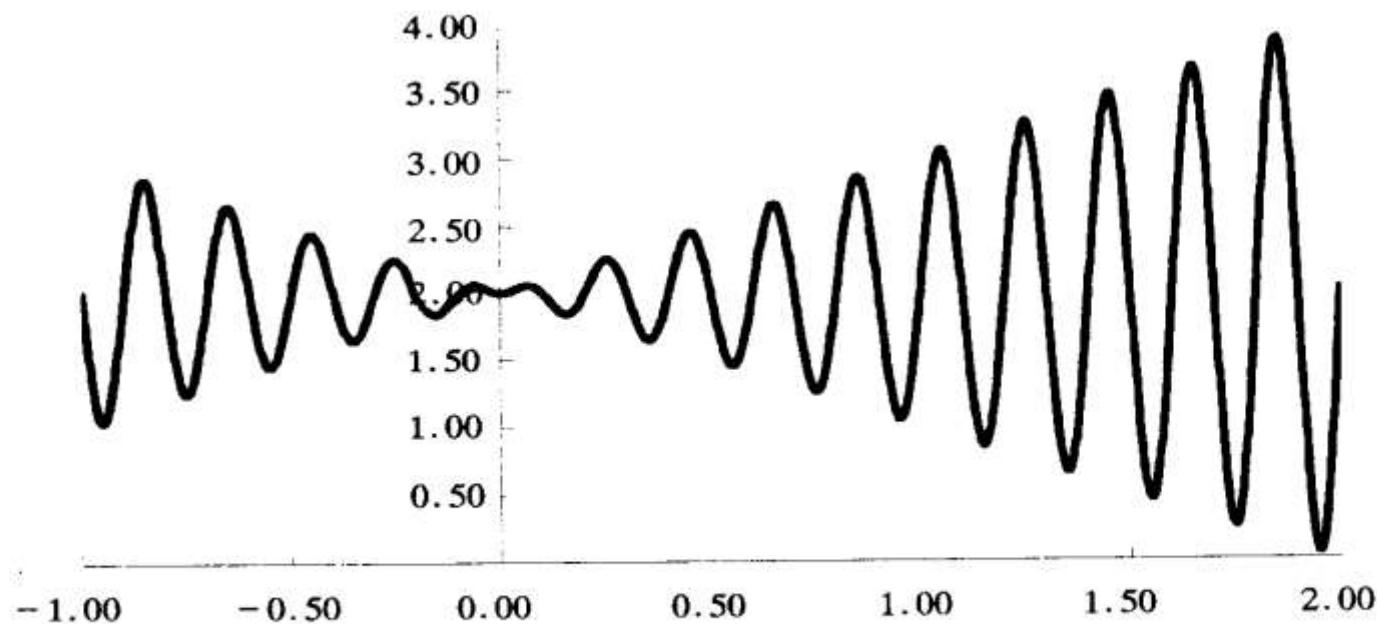


图 1 函数 $f(x) = x \sin(10\pi x) + 2.0$ 的曲线

例：约束单目标

$$\max f(x) = \left| \frac{\sum_{i=1}^n \cos^4(x_i) - 2 \prod_{i=1}^n \cos^2(x_i)}{\sqrt{\sum_{i=1}^n i x_i^2}} \right|$$

Subject to:

$$g_1(\bar{x}) = 0.75 - \prod_{i=1}^n x_i \leq 0$$

$$g_2(\bar{x}) = \sum_{i=1}^n x_i - 7.5n \leq 0 \quad ,$$

其中 $n = 20$, $0 \leq x_i \leq 10 (i = 1, \dots, n)$

$$f(x^*) = 0.803619$$

例：约束单目标

模糊 C-均值聚类是通过最小化关于隶属度矩阵 U 和聚类中心集合 V 的目标函数 $J_m(U, V)$ 来实现的,即

$$\min J_m(U, V) = \sum_{i=1}^n \sum_{k=1}^c (\mu_{i,k})^m d_{i,k}^2$$

$U = \{\mu_{i,k}\} \in \mathbf{R}^{n \times c}$ 满足以下条件:

① $\mu_{i,k} \in [0, 1], \forall i, k$;

② $\sum_k \mu_{i,k} = 1, \forall i$;

③ $0 < \sum_i \mu_{i,k} < n, \forall k$

例：多目标优化

- Function ZDT1 (convex, uniform Pareto Front): $f_1 = x_1$,

$$g = 1 + 9\left(\sum_{i=2}^n x_i\right)/(n-1), \quad f_2 = g(1 - \sqrt{f_1/g}). \quad n = 30, x_i \in [0,1].$$

- Function ZDT2 (convex, Pareto Front): $f_1 = x_1$, $g = 1 + 9\left(\sum_{i=2}^n x_i\right)/(n-1)$,

$$f_2 = g(1 - (f_1/g)^2). \quad n = 30, x_i \in [0,1].$$

- Function ZDT3 (Pareto Front that consists of separated convex parts): $f_1 = x_1$,

$$g = 1 + 9\left(\sum_{i=2}^n x_i\right)/(n-1), \quad f_2 = g(1 - \sqrt{f_1/g} - (f_1/g)\sin(10\pi f_1)). \quad n = 30, x_i \in [0,1].$$

- Function ZDT4 (Pareto Front with multimodality) $f_1 = x_1$, $f_2 = g(1 - \sqrt{f_1/g})$,

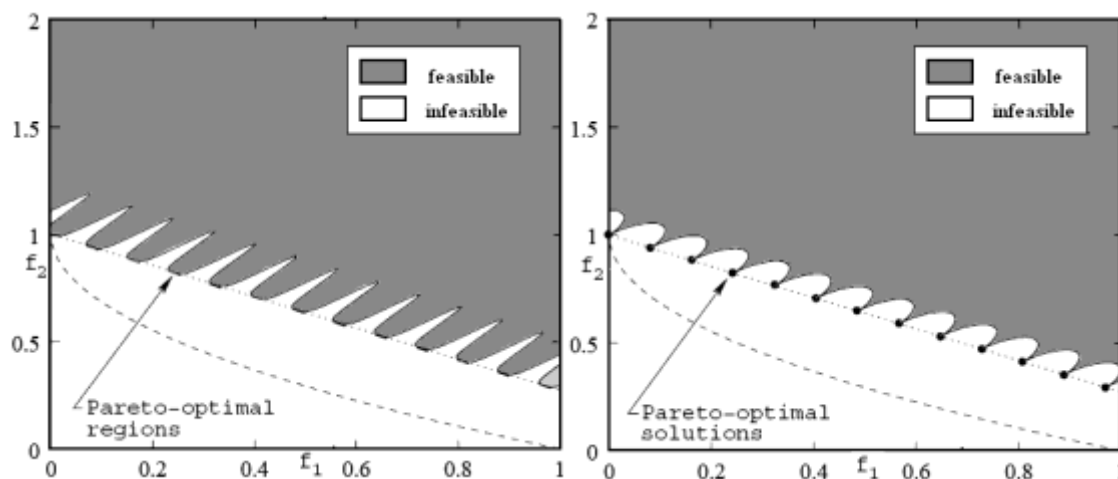
$$g = 1 + 10(n-1) + \sum_{i=2}^n (x_i^2 - 10\cos(4\pi x_i)). \quad n = 10, x_1 \in [0,1], x_2, \dots, x_n \in [-5,5].$$

- Function ZDT6 (nonconvex, non-uniform Pareto Front): $f_1 = 1 - \exp(-4x_1)\sin^6(6\pi x_1)$

例：约束多目标

$$\left\{ \begin{array}{l} \min F(x) = \min \left\{ \begin{array}{l} f_1(x) \\ f_2(x) \end{array} \right\} = \min \left\{ \begin{array}{l} x_1 \\ c(\bar{x}) \left[1 - \frac{f_1(x)}{c(x)} \right] \end{array} \right\} \\ \cos(\theta)[f_2(x) - e] - \sin(\theta)f_1(x) \geq a \mid \sin\{b\pi[\sin(\theta)[f_2(x) - e] + \cos(\theta)f_1(x)]^c\} |^d \end{array} \right.$$

$$c(x) = 41 + \sum_{i=2}^5 [x_i^2 - 10 \cos(2\pi x_i)], \quad 0 \leq x_1 \leq 1, -5 \leq x_i \leq 5, i = 2, 3, 4, 5$$



组合优化问题

- 组合优化问题的决策变量是离散取值的
- 很多离散组合优化问题都是从运筹学（Operations Research, OR）中演化出来的
- 组合优化其所研究的问题涉及到信息技术、经济管理、工业工程、交通运输、通信网络等众多领域，在科学研究和生产实践中都起着重要的作用

组合优化问题

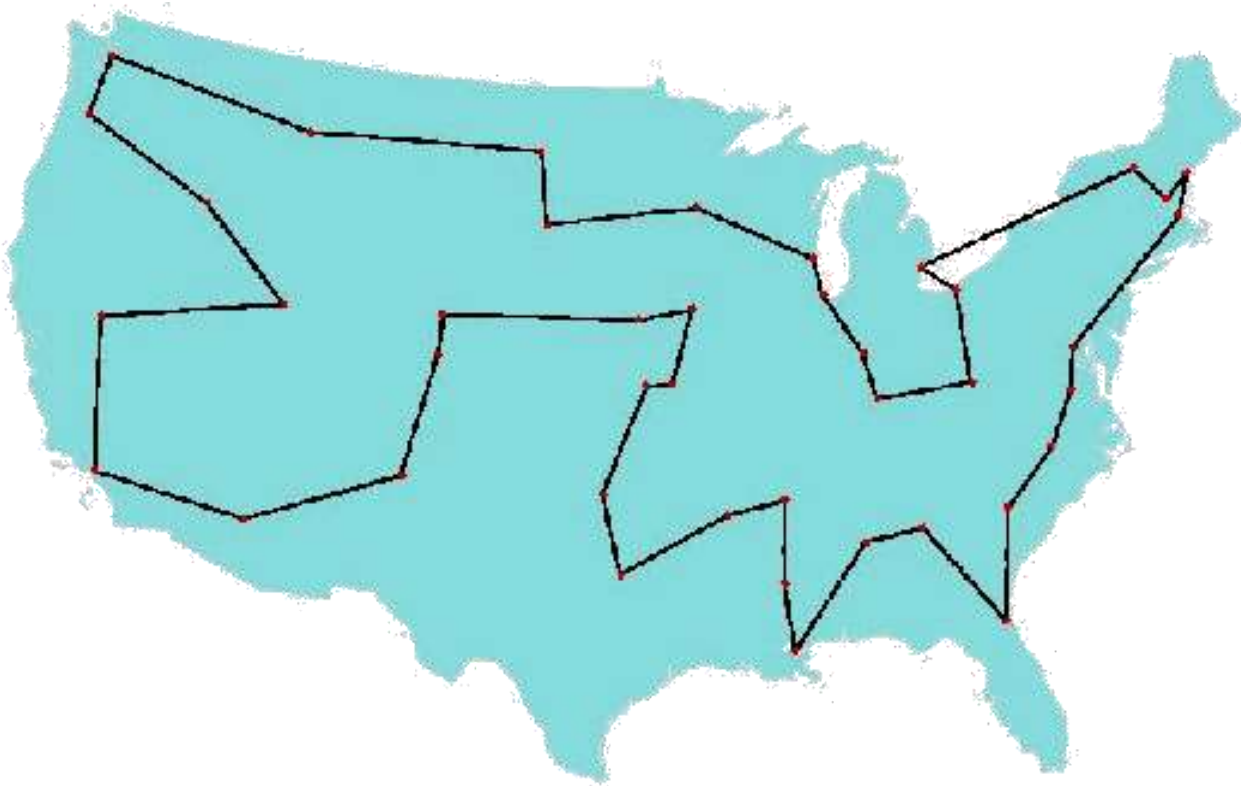
- 经典组合优化问题：
 - 旅行商问题 (Traveling Salesman Problem, TSP)
 - **0-1**背包问题 (Zero/one Knapsack Problem, ZKP/0-1KP/KP)
 -
- 当问题规模 n 比较大时，用枚举方法所需时间太大，我们借助智能优化计算方法，可以在合理的时间内求解得到令人满意的解，从而满足实践的需要

例：离散组合优化TSP

旅行商问题（TSP, traveling salesman problem）

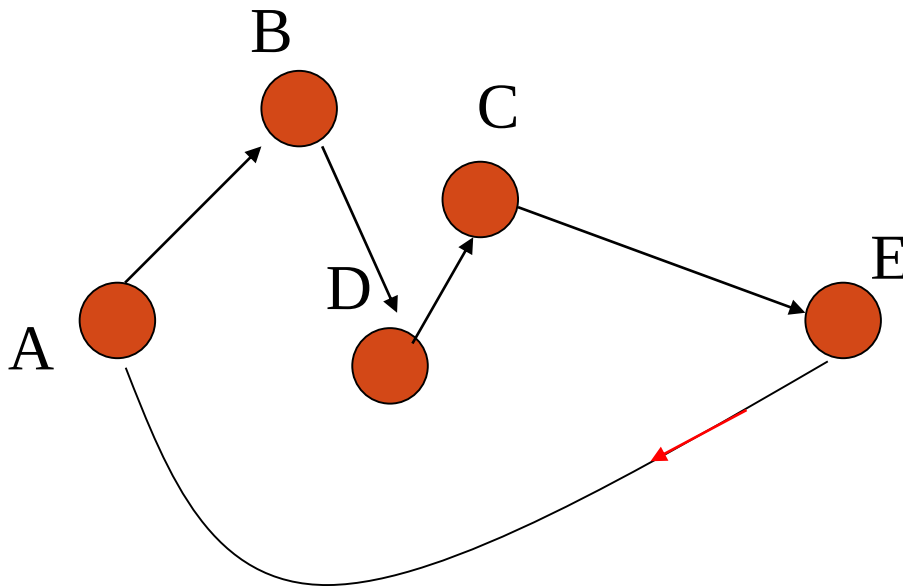
管梅谷教授1960年首先提出，国际上称之为中国邮递员问题。

问题描述：一商人去 n 个城市销货，所有城市走一遍再回到起点，使所走路程最短。



TSP

The one below is length: 33



	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

例：0-1背包问题

现在有四种物品，它们的重量与价值分别给出。有一个背包，总容量为8。问怎么选取物品，可以使得背包装的物品价值最大？

(1) 0-1背包问题

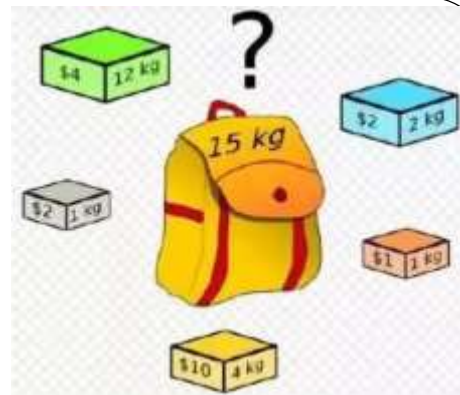
物品编号	物品重量	物品价值	物品数量
1	2	3	1
2	3	4	1
3	4	5	1
4	5	8	1

(2) 多重背包问题

物品编号	物品重量	物品价值	物品数量
1	2	3	2
2	3	4	2
3	4	5	2
4	5	8	2

(3) 完全背包问题

物品编号	物品重量	物品价值	物品数量
1	2	3	无数件
2	3	4	无数件
3	4	5	无数件
4	5	8	无数件



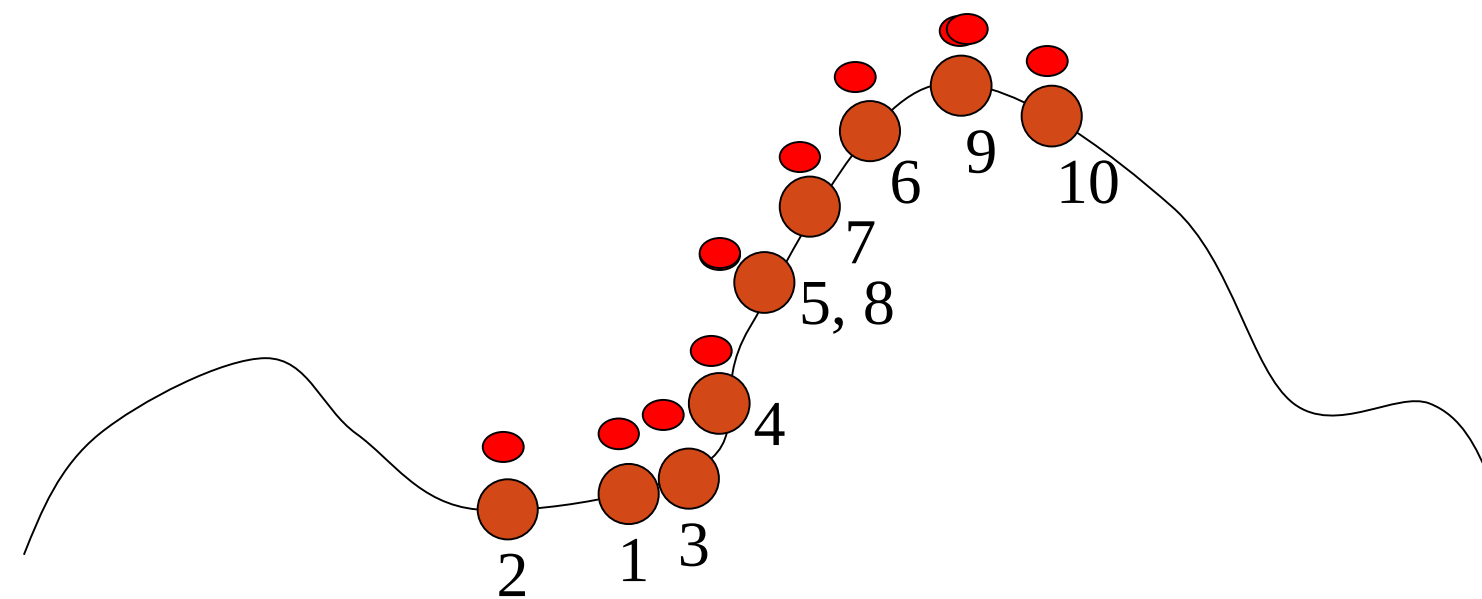
传统最优化面临新挑战：实际问题

- 离散性问题：组合爆炸
- 不确定性问题：随机性数学模型
- 大规模问题：超高维
- 动态优化问题

现代优化方法：

- 追求满意—近似解
- 实用性强—解决实际问题

随机算法：爬山法

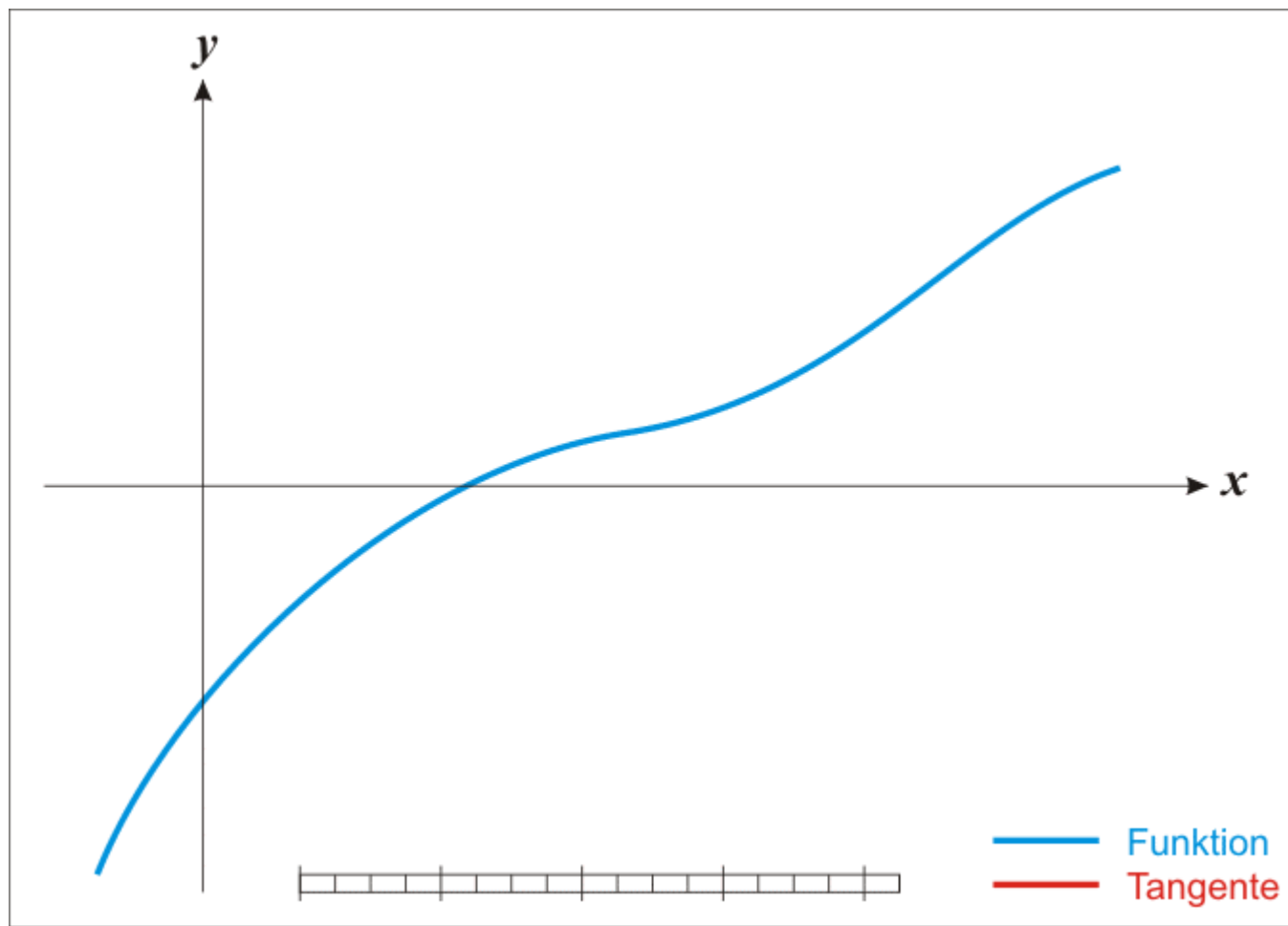


牛顿法

- Min $f(x)$
- 求解方程 $f(x) = 0$
- 计算穿过点 $(x_0, f(x_0))$ 并且斜率为 $f'(x_0)$ 的直线和 x 轴的交点的 x 坐标

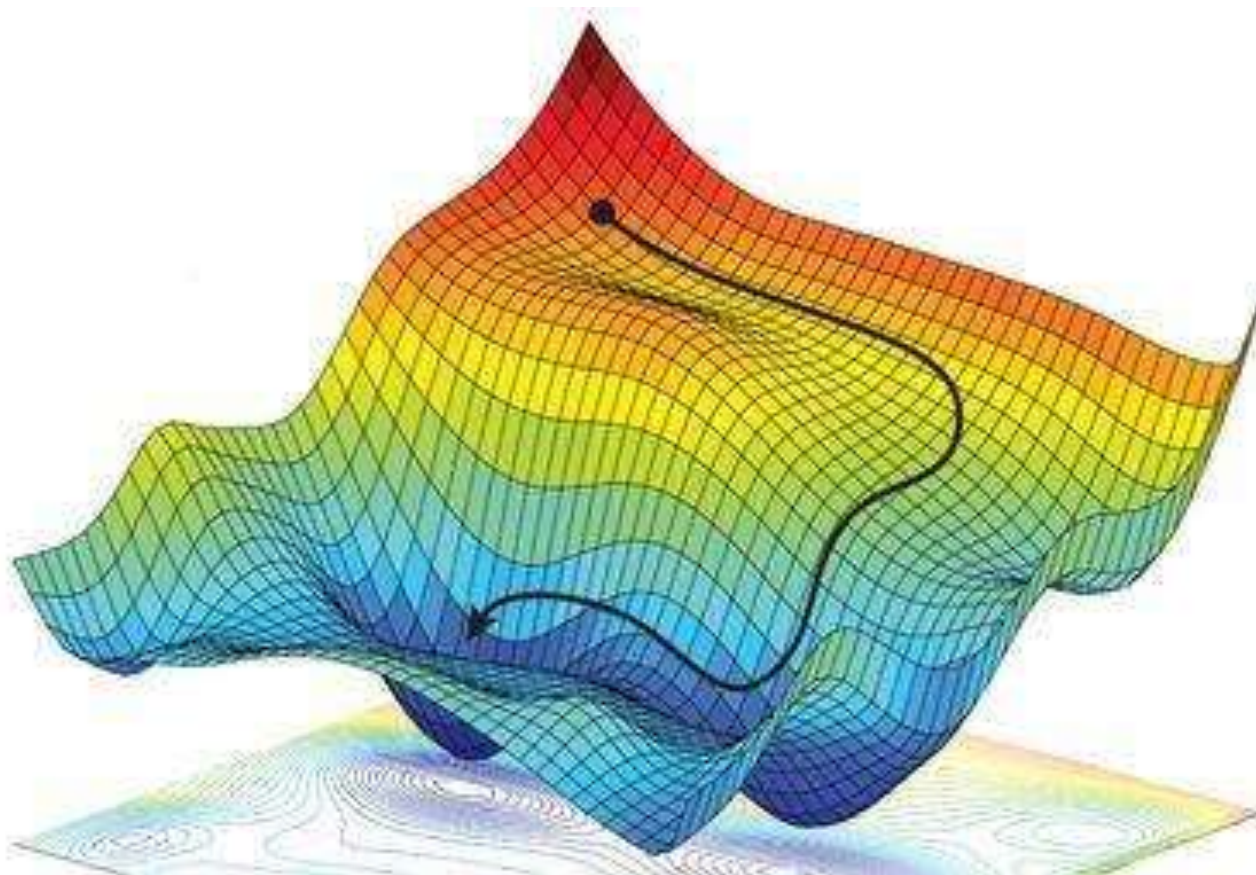
$$x \cdot f'(x_0) + f(x_0) - x_0 \cdot f'(x_0) = 0$$

牛顿法

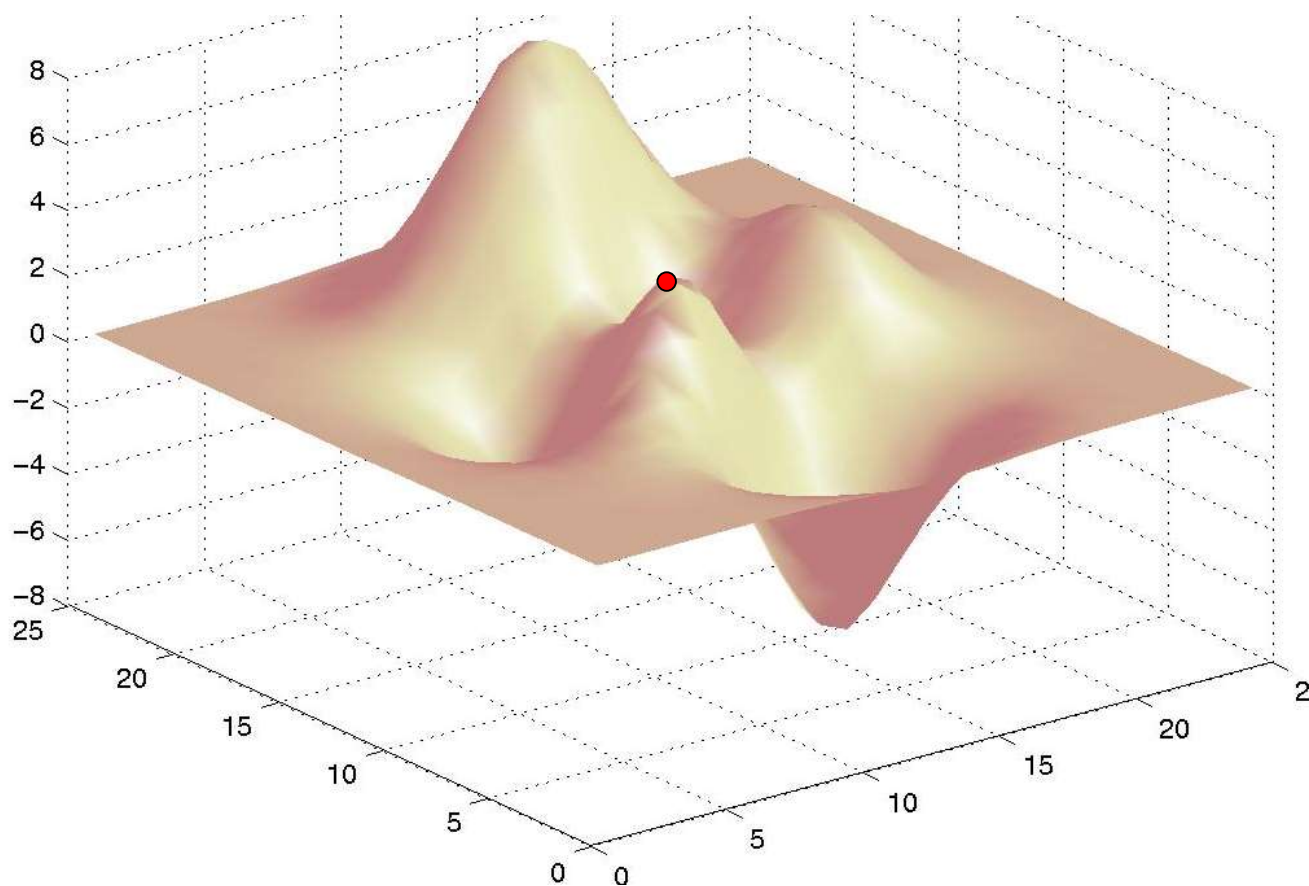


$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

梯度下降



易陷入局部最优



计算复杂性

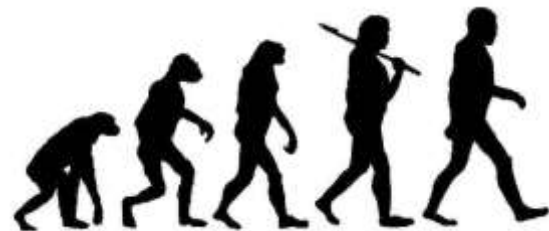
- 计算复杂性（Computational Complexity）：描述求解问题的难易程度或者算法的执行效率
- 对于算法的计算复杂性，我们一般很容易进行判断，例如使用蛮力法去枚举旅行商问题或者0-1背包问题的算法，就是具有指数计算复杂性的算法
- 对于某问题的计算复杂性进行判断却不是一件简单的事情

计算复杂性

- 问题的计算复杂性是问题规模的函数，故需要首先定义问题的规模
- 例如对于矩阵运算，矩阵的阶数可被定义为问题的规模
 - 如果求解一个问题需要的运算次数或步骤数是问题规模 n 的指数函数，则称该问题有指数时间复杂性
 - 如果所需的运算次数是 n 的多项式函数，则称它有多项式时间复杂性
- 对于某个具体问题，其复杂性上界是已知求解该问题的最快算法的复杂性，而复杂性下界只能通过理论证明来建立
- 证明一个问题的复杂性下界就需要证明不存在任何复杂性低于下界的算法。显然，建立下界要比确定上界困难得多

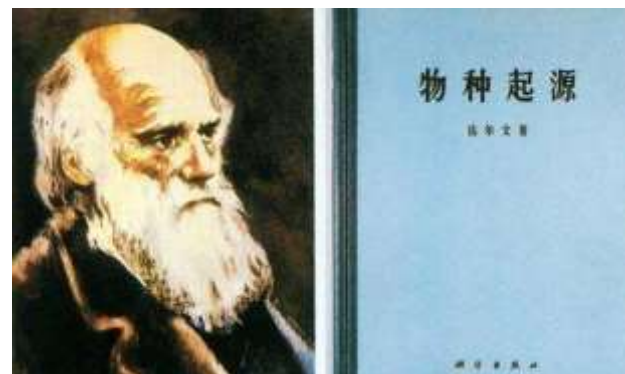
第二节：进化论

Lamarck 进化论



- 一切物种都是其他物种演变和进化而来的，而生物的演变和进化是一个缓慢和连续的过程。
- 环境的变化能够引起生物的变异，环境的变化迫使生物发生适应性的进化。有神经系统的动物发生变异的原因，除了环境变化和杂交外，更重要是通过用进废退和获得性状遗传。
- 生物进化的方向从简单到复杂，从低到高。最原始的生物起源于自然发生。生物并不起源于共同祖先。
- 拉马克曾以长颈鹿的进化为例，说明他的“用进废退”观点。长颈鹿的祖先颈部并不长，由于干旱等原因。在低处已找不到食物，迫使它伸长脖颈去吃高处的树叶，久而久之，它的颈部就变长了。一代又一代，遗传下去，它的脖子越来越长，终于进化为现在所见的长颈鹿。

Darwin 进化论



- 包括两部分：遗传(基因)变异；自然选择
- 生物不是静止的，而是进化的。物种不断变异，旧物种消灭，新物种产生。生物的进化是连续和逐渐，不会发生突变。
- 生物之间存在一定的亲缘关系，**他们具有共同的祖先。**
- 自然选择。生物过度繁殖，但是它们的生存空间和食物有限，从而面临生存斗争，包括：种内、种间以及生物与环境的斗争。**自然选择是达尔文进化论的核心。**

孟德尔学说



- 1857年，身为神父的孟德尔通过对植物进行一系列仔细的实验。
- 孟德尔发现植物的父母单独地把自身的性状传递给子。他还发现性状不是单纯地混合在一起，而是保持着独立性：一株高的植物和一株矮的植物繁殖出来的后代要么是高的，要么是矮的，而不是介于两者之间。
- 表明性状是分开独立地遗传给下一代，后来这称为遗传基因。
- 孟德尔的重要发现被人们忽略了数十年，直到20世纪初才被人们认可。

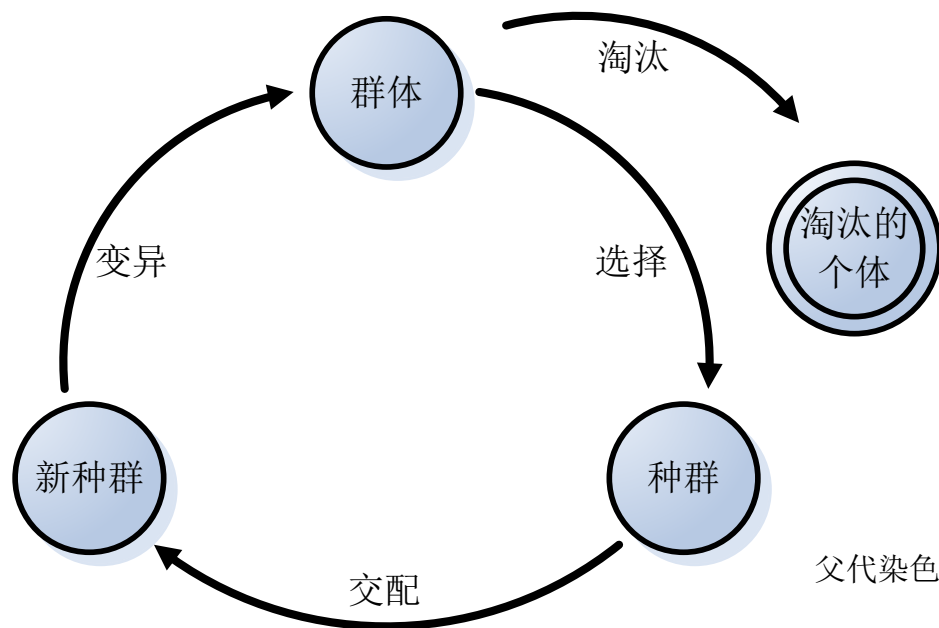
孟德尔学说

- 分离定律： 基因作为独特的独立单位而代代相传。细胞中有成对的基本遗传单位，在杂交的生殖细胞中，一个来自雄性亲本，一个来自雌性亲本。
- 独立分配定律： 在一对染色体上的基因对中的等位基因能够独立遗传。
- 孟德尔的这两条遗传基本定律就是新遗传学的起点，孟德尔也因此被后人称为现代遗传学的奠基人。

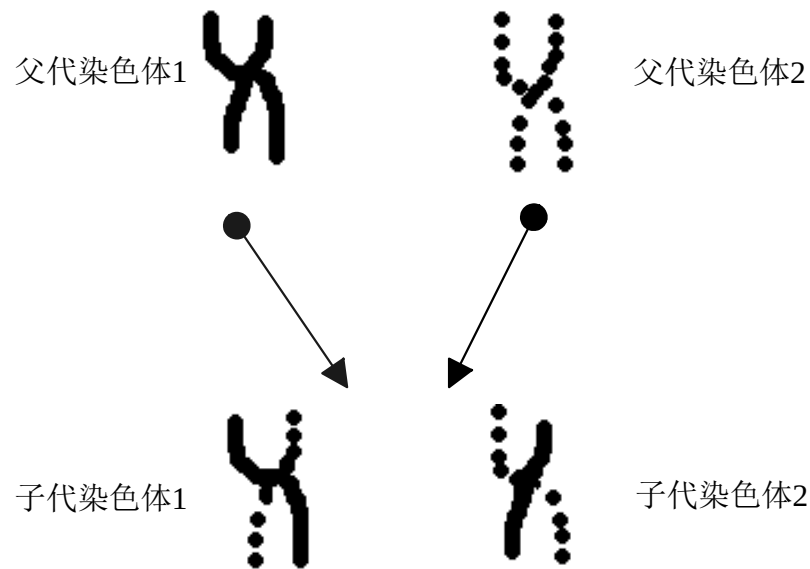
新达尔文主义

- 将达尔文进化论和孟德尔-摩根基因相结合，成为现被广泛接受的新达尔文主义。
- 新达尔文主义认为，只用种群上和物种内的少量统计过程就可以充分地解释大多数生命历史，这些过程就是繁殖、变异、竞争、选择。
- 繁殖是所有生命的共同特性；变异保证了任何生命系统能在正熵世界中连续繁殖自己；对于限制在有限区域中的不断膨胀的种群来说，竞争和选择是不可避免的结论。

遗传基因重组过程



生物进化过程



生物学中的遗传概念

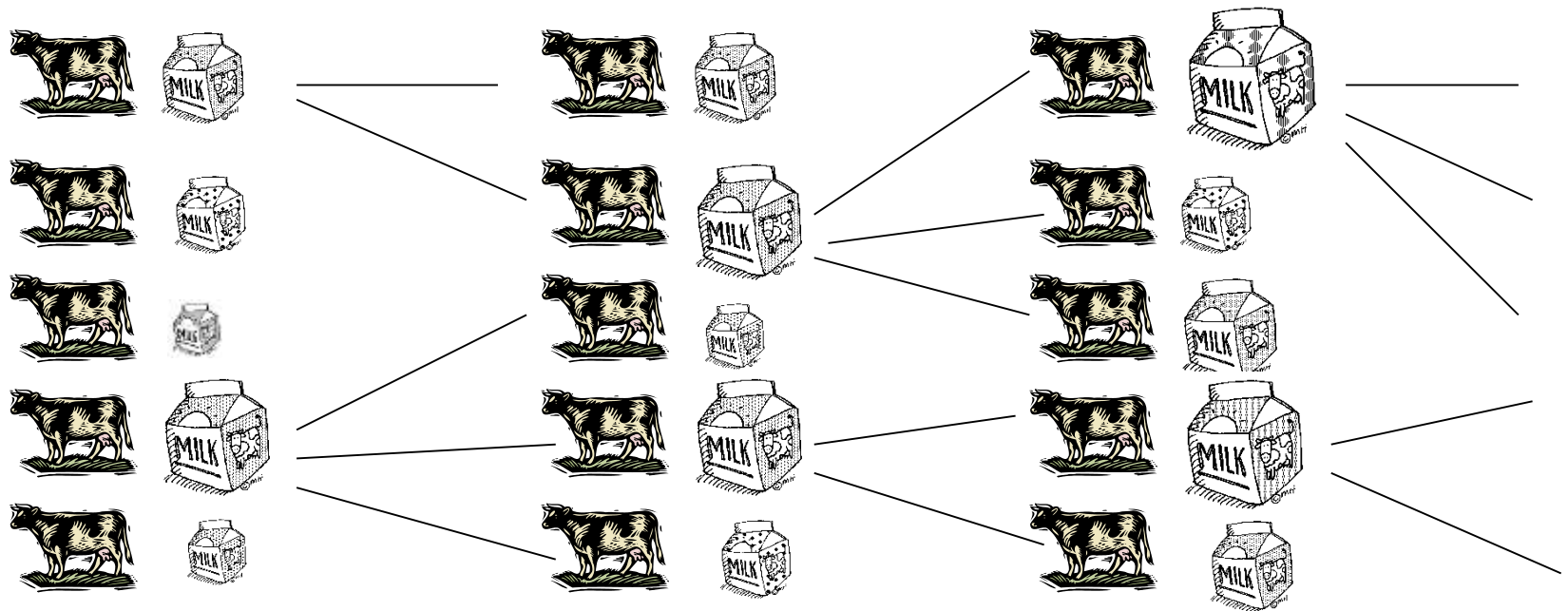
- 在生物细胞中，控制并决定生物遗传特性的物质是脱氧核糖核酸，简称DNA。染色体是其载体。
- DNA是由四种碱基按一定规则排列组成的长链。四种碱基不同的排列决定了生物不同的表现性状。例如，改变DNA长链中的特定一段（称为基因），即可改变人体的身高。
- 细胞在分裂时，DNA通过复制而转移到新产生的细胞中，新的细胞就继承了上一代细胞的基因。

生物学中的遗传概念

- 有性生殖生物在繁殖下一代时，两个同元染色体之间通过交叉而重组，亦即在两个染色体的某一相同位置处DNA被切断，其前后两串分别交叉形成两个新的染色体。
- 在细胞进行复制时可能以很小的概率产生某些复制差错，从而使DNA发生某种变异，产生新的染色体。
- 这些新的染色体将决定新的个体（后代）的新的性状。

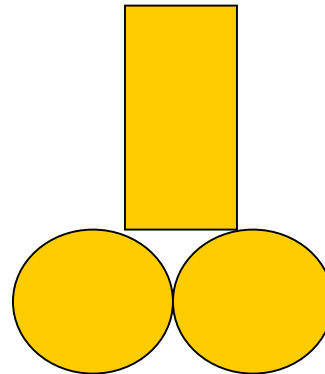
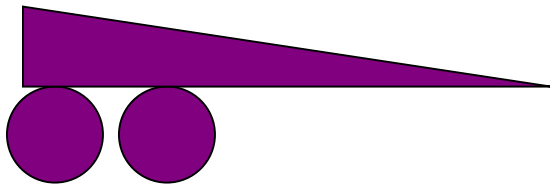
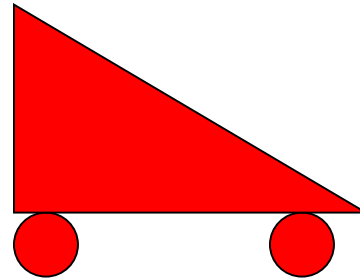
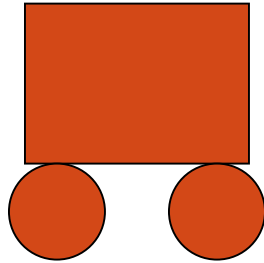
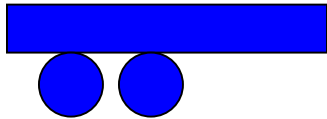
生物学中的遗传概念

- 在一个群体中，并不是所有的个体都能得到相同的繁殖机会，对生存环境适应度高的个体将获得更多的繁殖机会；对生存环境适应度较低的个体，其繁殖机会相对较少，即所谓自然选择。而生存下来的个体组成的群体，其品质不断得以改良，称为进化。
- 生物的进化是经过无数次有利的进化积累而成的，不同的环境保留了不同的变异后代！

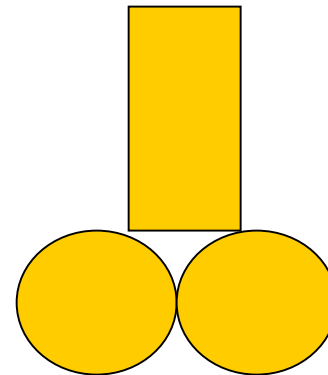
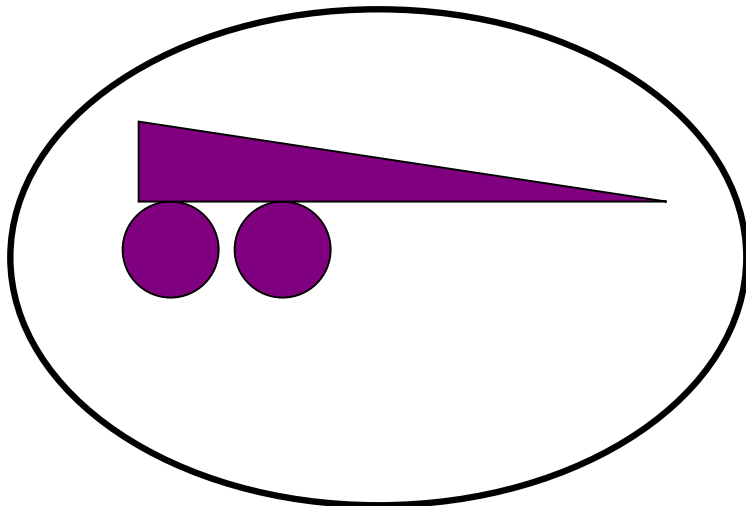
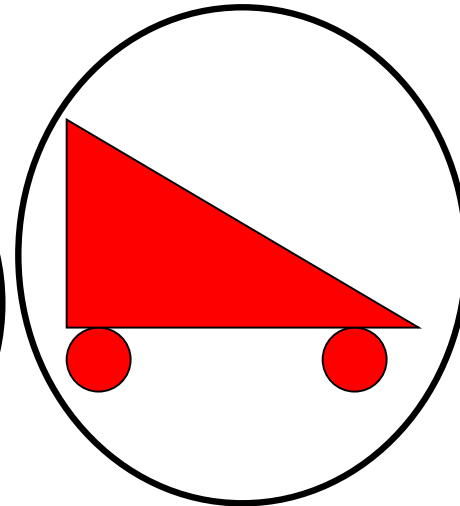
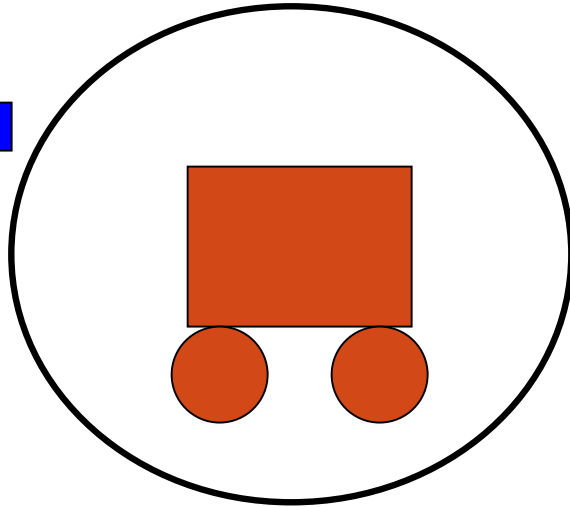
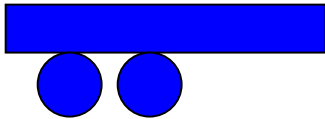


Time

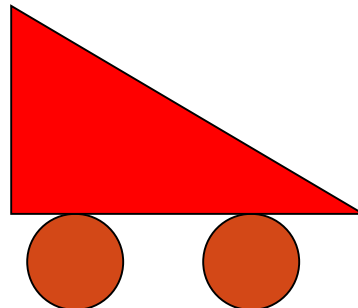
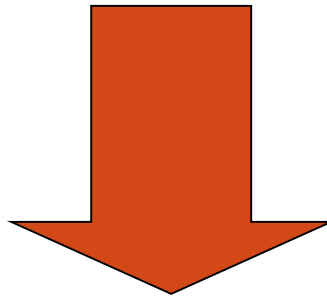
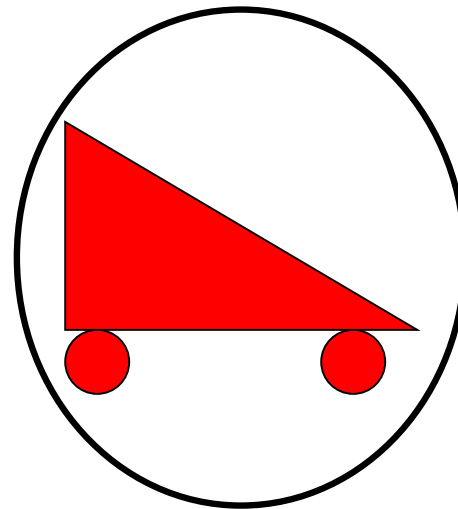
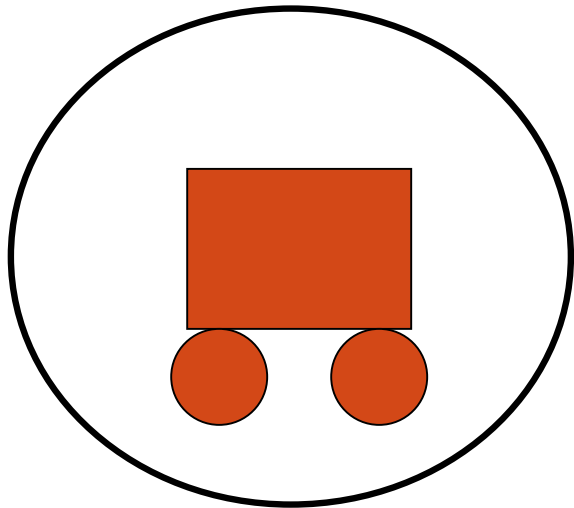
Initial population



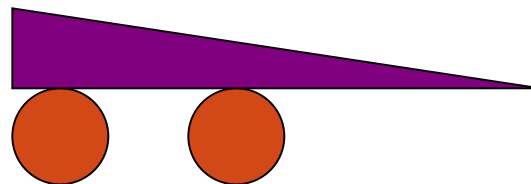
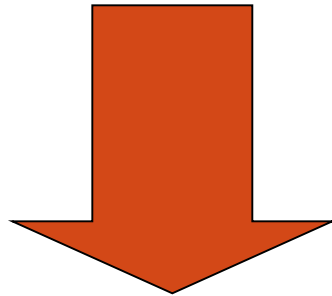
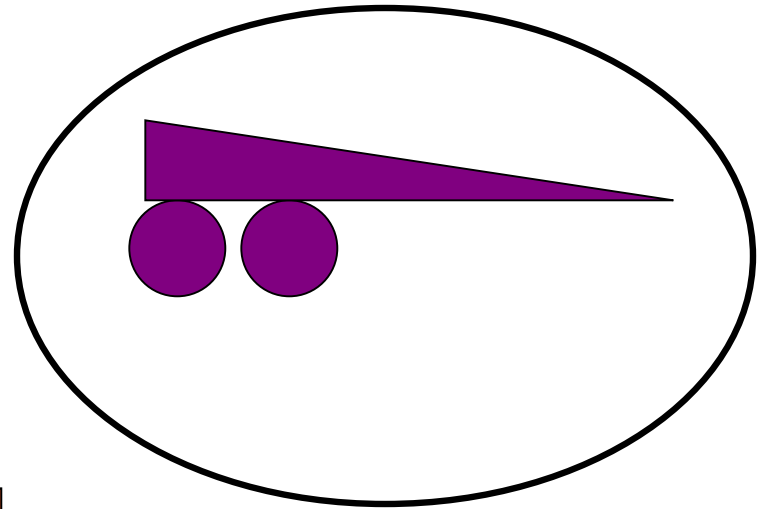
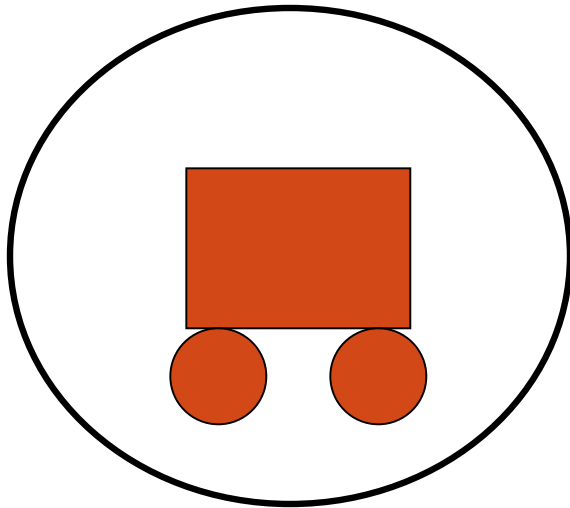
Select



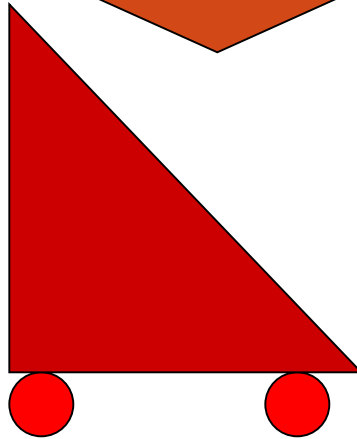
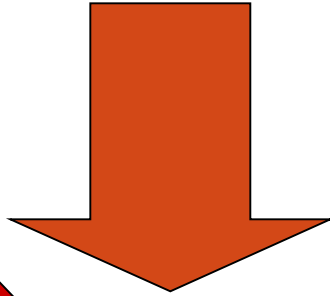
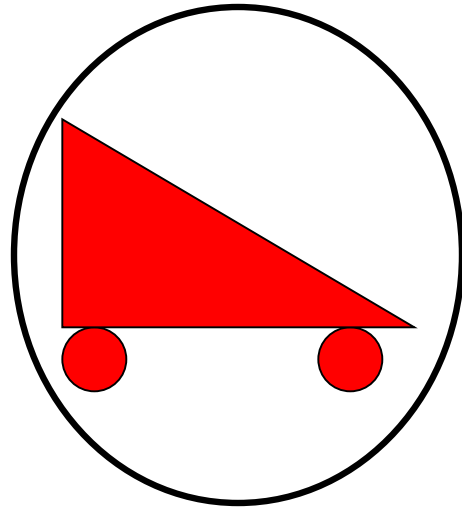
Crossover



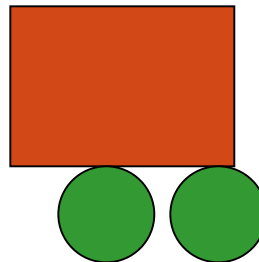
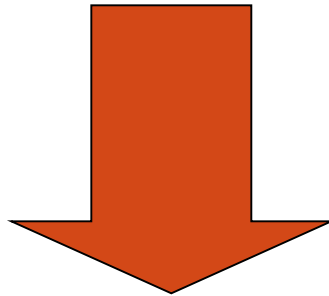
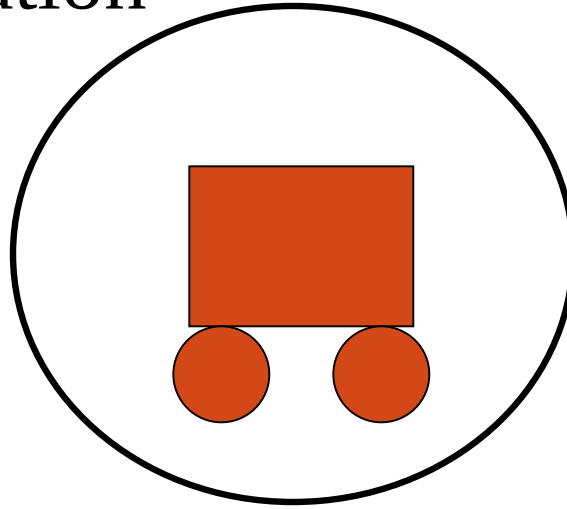
Another Crossover



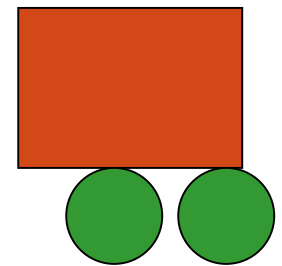
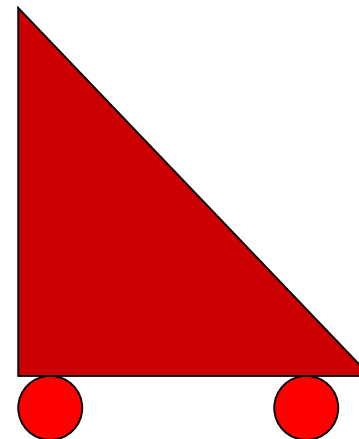
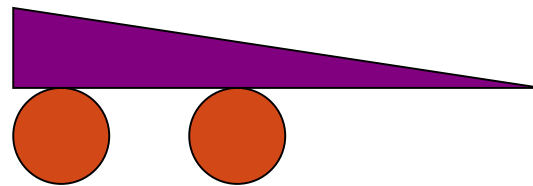
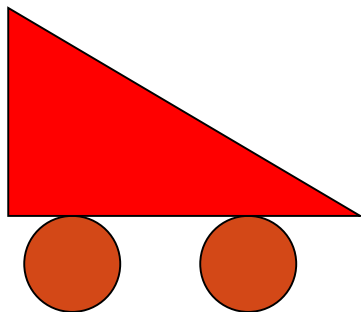
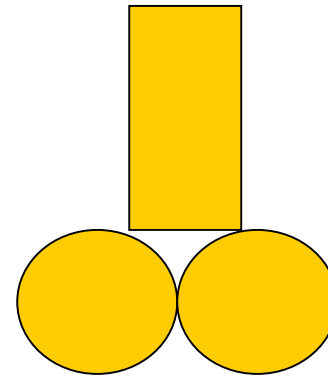
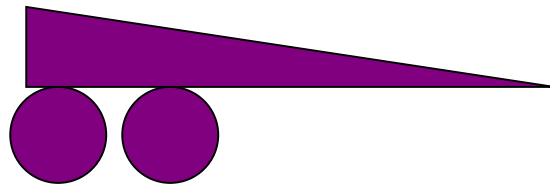
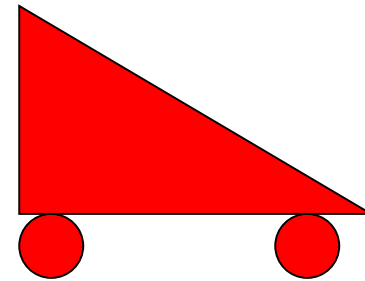
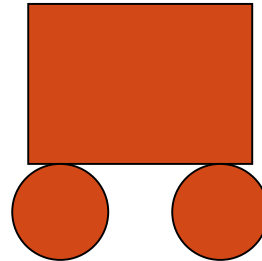
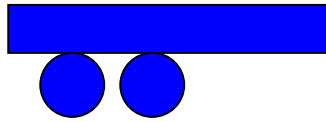
A mutation



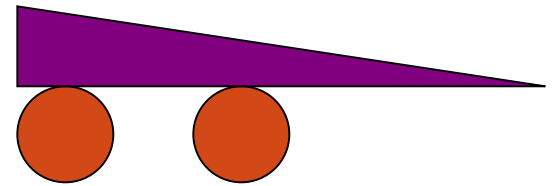
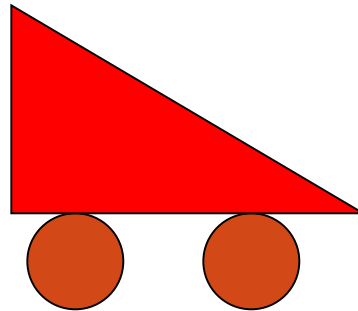
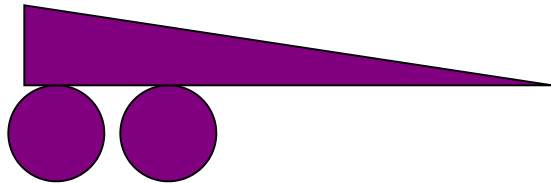
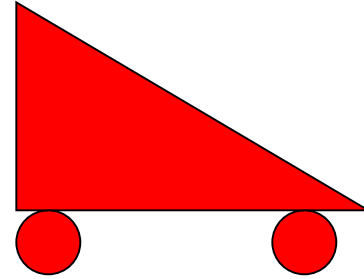
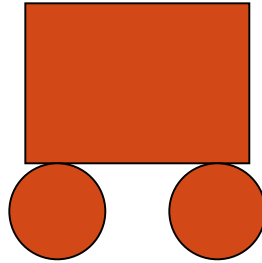
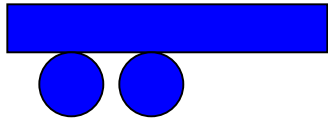
Another Mutation



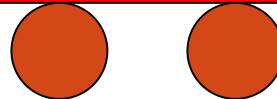
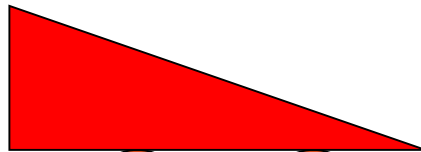
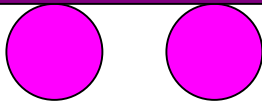
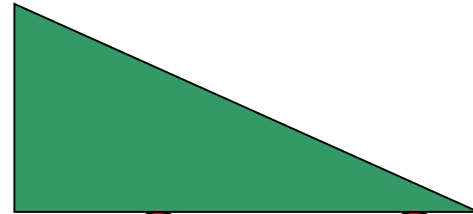
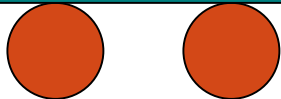
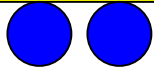
Old population + children



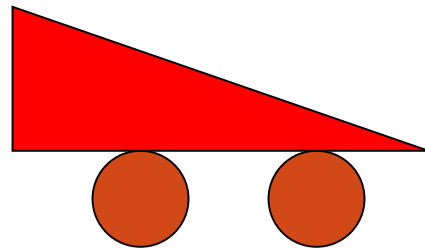
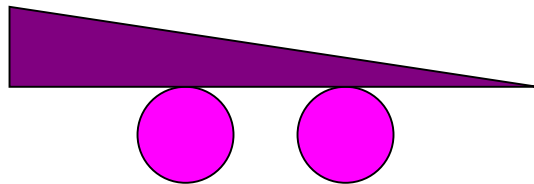
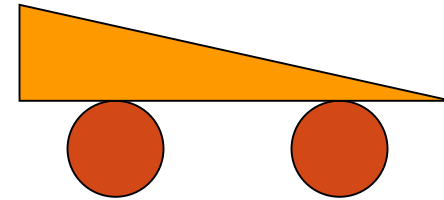
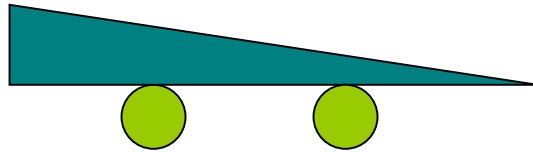
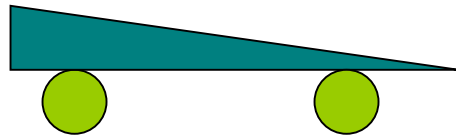
New Population: Generation 2



Generation 3



Generation 4, etc ...



第三节：遗传算法思想

概念说明

选择：通过适应度的计算，淘汰不合理的个体。类似于自然界的物竞天择。

复制：编码的拷贝，类似于细胞分裂中染色体的复制。

交叉：编码的交叉重组，类似于染色体的交叉重组。

变异：编码按小概率扰动产生的变化，类似于基因的突变。

这个过程将导致种群像自然进化一样，后代种群比前代更加适应环境，末代种群中得最优个体经过解码（从基因到性状的映射），可以作为问题近似最优解。

生物遗传进化过程

生物遗传进化

- 群体
- 种群
- 染色体
- 基因
- 适应能力
- 交叉
- 变异
- 进化结束

类比关系

遗传算法

遗传算法

- 搜索空间的一组有效解
- 选择得到的新群体
- 可行解的编码串
- 染色体的一个编码单元
- 染色体的适应值
- 染色体交换部分基因得到新染色体
- 染色体某些基因的数值改变
- 算法结束

遗传算法历史

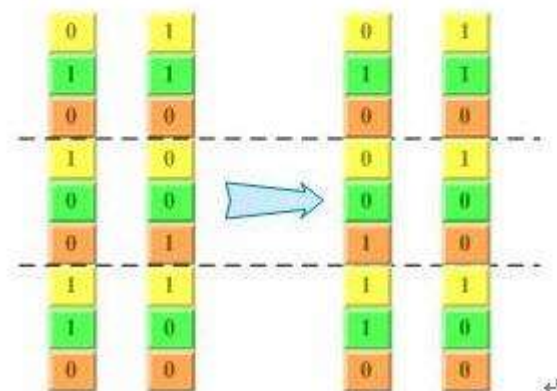
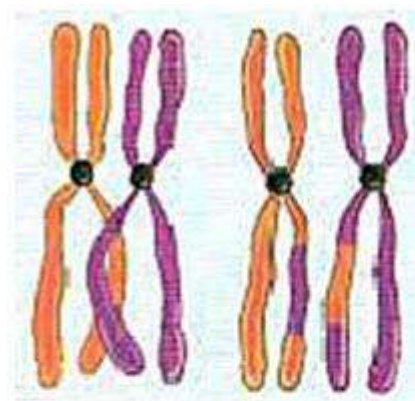
- 1965年，Holland首次提出人工遗传操作的重要性，并把这些应用于自然系统和人工系统中。
- 1967年，Bagley在他的论文中首次提出了遗传算法这一术语，并讨论了遗传算法在自动博弈中的应用。
- 1970年，Cavichio把遗传算法应用于模式识别中。第一个把遗传算法应用于函数优化的是Hollstien。

遗传算法历史

- 1975年Holland出版了他的著名专著《自然系统和人工系统的适应性》该书系统地阐述了遗传算法的基本理论和方法，并提出了对遗传算法极为重要的**模式理论**，该理论首次确认了结构重组遗传操作对于获得隐并行性的重要性。
- 同年，DeJong完成了他的重要论文《遗传自适应系统的行为分析》。他在该论文中所做的研究工作可看作是遗传算法发展过程中的一个里程碑。

遗传算法历史

- 在一系列研究工作的基础上，上世纪80年代由Goldberg进行归纳总结，形成了遗传算法的基本框架。



著名的遗传算法类型

- **Genetic Algorithms (GA)**, developed by Dr. Holland.
Holland, J.: *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, MI, 1975; MIT Press, Cambridge, MA, 1992.
Goldberg, D.: *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, Reading, MA, 1989.
- **Evolution Strategies (ES)**, developed by Dr. Rechenberg and Dr. Schwefel.
Rechenberg, I.: *Evolution strategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*, Frommann-Holzboog, 1973.
Schwefel, H.: *Evolution and Optimum Seeking*, John Wiley & Sons, 1995.
- **Evolutionary Programming (EP)**, developed by Dr. Fogel.
Fogel, L. A. Owens & M. Walsh: *Artificial Intelligence through Simulated Evolution*, John Wiley & Sons, 1966.
- **Genetic Programming (GP)**, developed by Dr. Koza.(遗传编程)
Koza, J. R.: *Genetic Programming*, MIT Press, 1992.
Koza, J. R.: *Genetic Programming II*, MIT Press, 1994.

与GA相关的重要学术期刊与国际会议

● 重要学术期刊

- Evolutionary Computation
- IEEE Transactions on Evolutionary Computation
-

● 重要国际会议

- International Conference on Genetic Algorithm
- ACM Genetic and Evolutionary Computation Conference
- Workshop on Foundations of Genetic Algorithms and Classifier Systems
- Genetic Programming Conference
- International Workshop on Artificial Life
-

搜索方法

```
graph TD; A[搜索方法] --> B[解析法]; A --> C[随机法]; A --> D[枚举法]; B --> E[直接法]; B --> F[间接法]; C --> G[导向随机]; C --> H[盲目随机]; D --> I[动态规划]; G --> J[Tabu搜索]; G --> K[模拟退火]; G --> L[演化算法]; G --> M[...];
```

解析法

随机法

枚举法

直接法

间接法

导向随机

盲目随机

动态规划

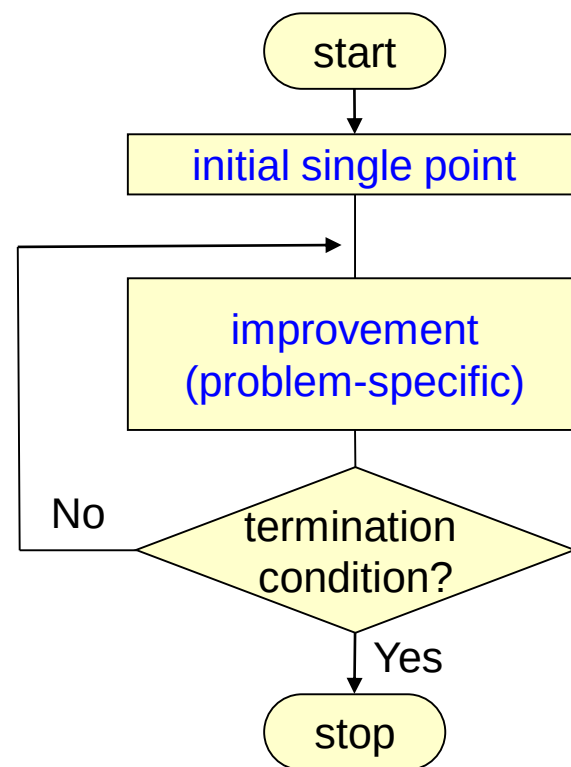
Tabu搜索 模拟退火 演化算法 ...

梯度下降优化方法

特点：

- 渐进收敛
- 基于梯度
- 单点搜索
- 局部最优

Conventional Method

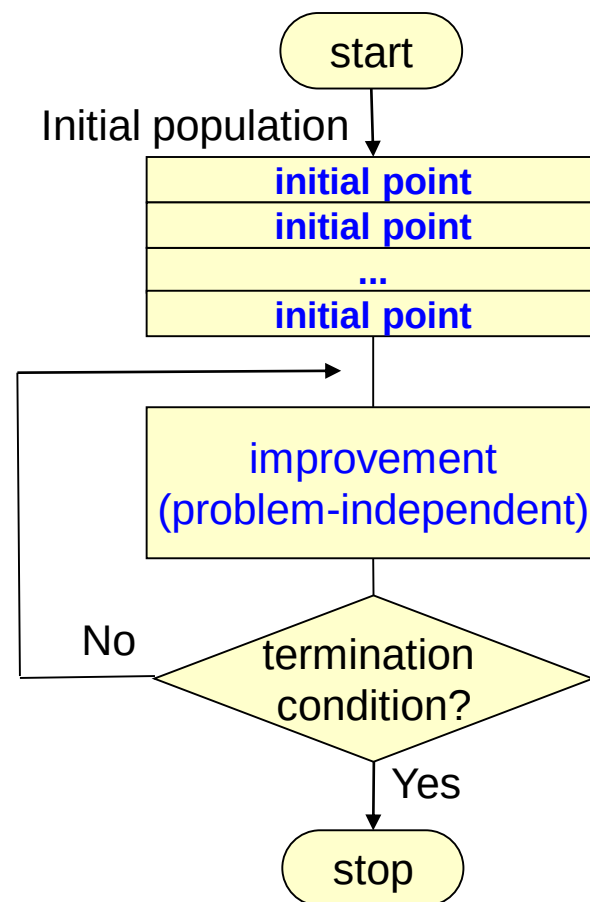


遗传算法

□传统的优化算法往往直接利用决策变量的实际值本身进行优化计算。

□遗传算法不是直接以决策变量的值，而是以**决策变量的某种形式的编码为运算对象**，从而可以很方便地引入和应用遗传操作算子。

Genetic Algorithm

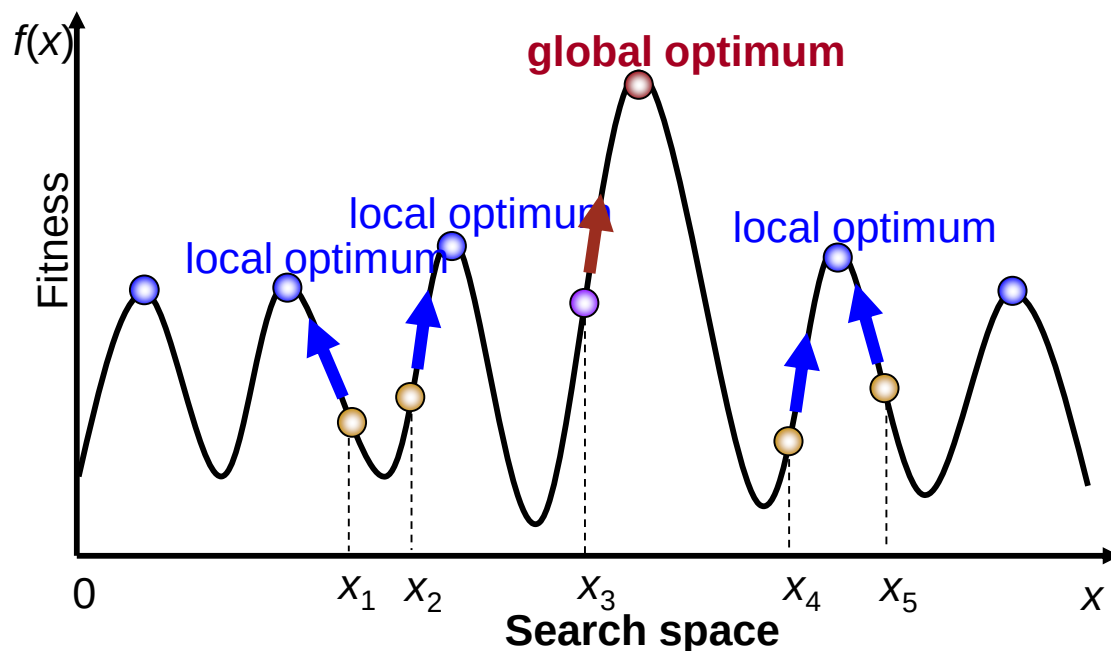


优点1

- 遗传算法直接以目标函数值作为搜索信息。
- 传统的优化算法往往不只需要目标函数值，还需要目标函数的导数等其它信息。
- 对许多目标函数无法求导或很难求导的函数，遗传算法就比较方便。

优点2

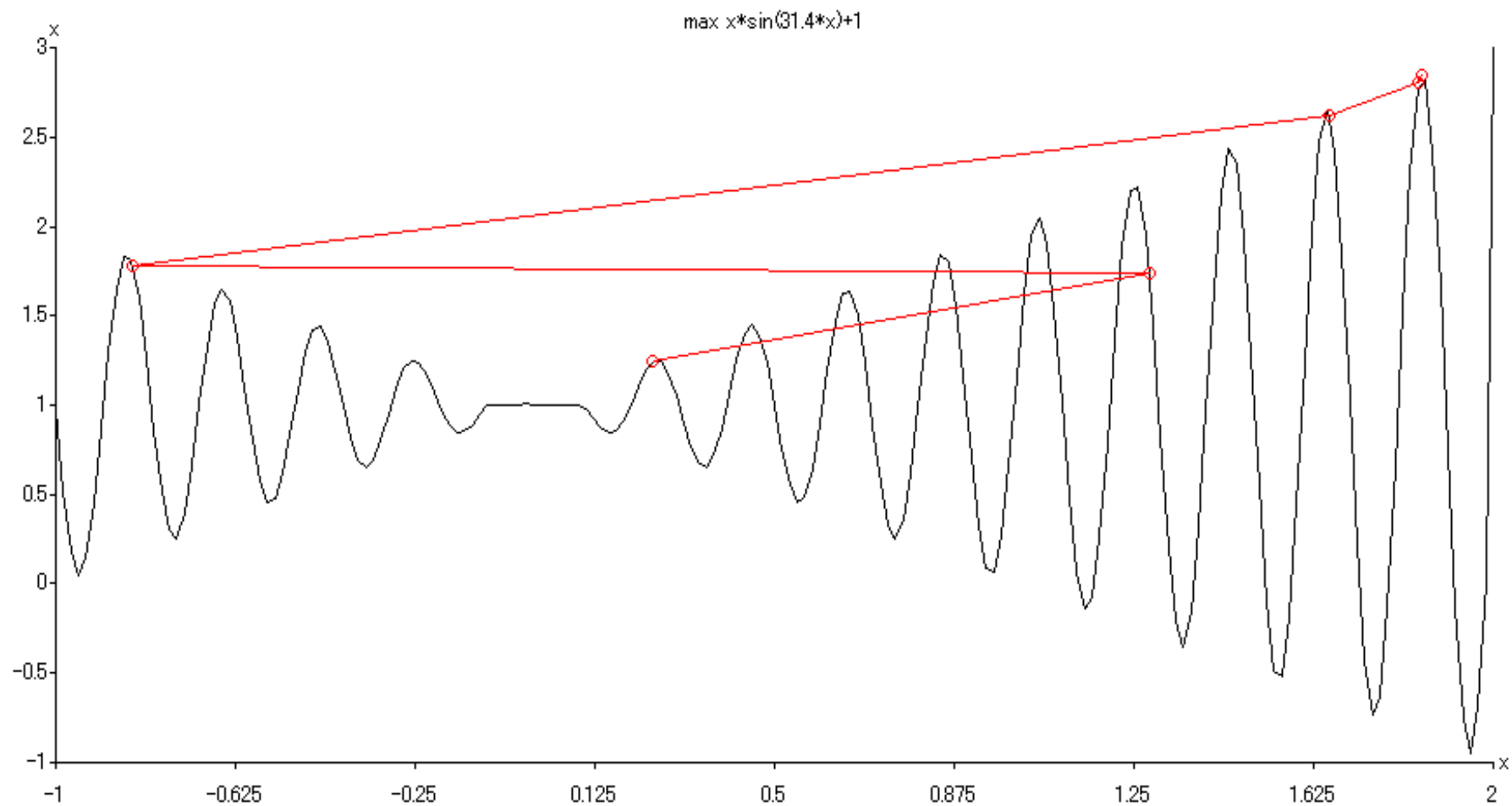
- 遗传算法同时进行解空间的**多点**搜索。传统的优化算法往往从解空间的一个初始点开始搜索，这样**容易陷入局部极值点**。
- 遗传算法进行群体搜索，而且在搜索的过程中引入遗传运算，使群体又可以不断进化。这些是遗传算法所特有的一种隐含**并行性**。



优点3

- 遗传算法属于一种 **自适应概率** 搜索技术，其选择、交叉、变异等运算都是以一种概率的方式来进行的，从而增加了其搜索过程的灵活性。
- 实践和理论都已证明了在一定条件下遗传算法总是以概率1收敛于问题的最优解。

搜索过程

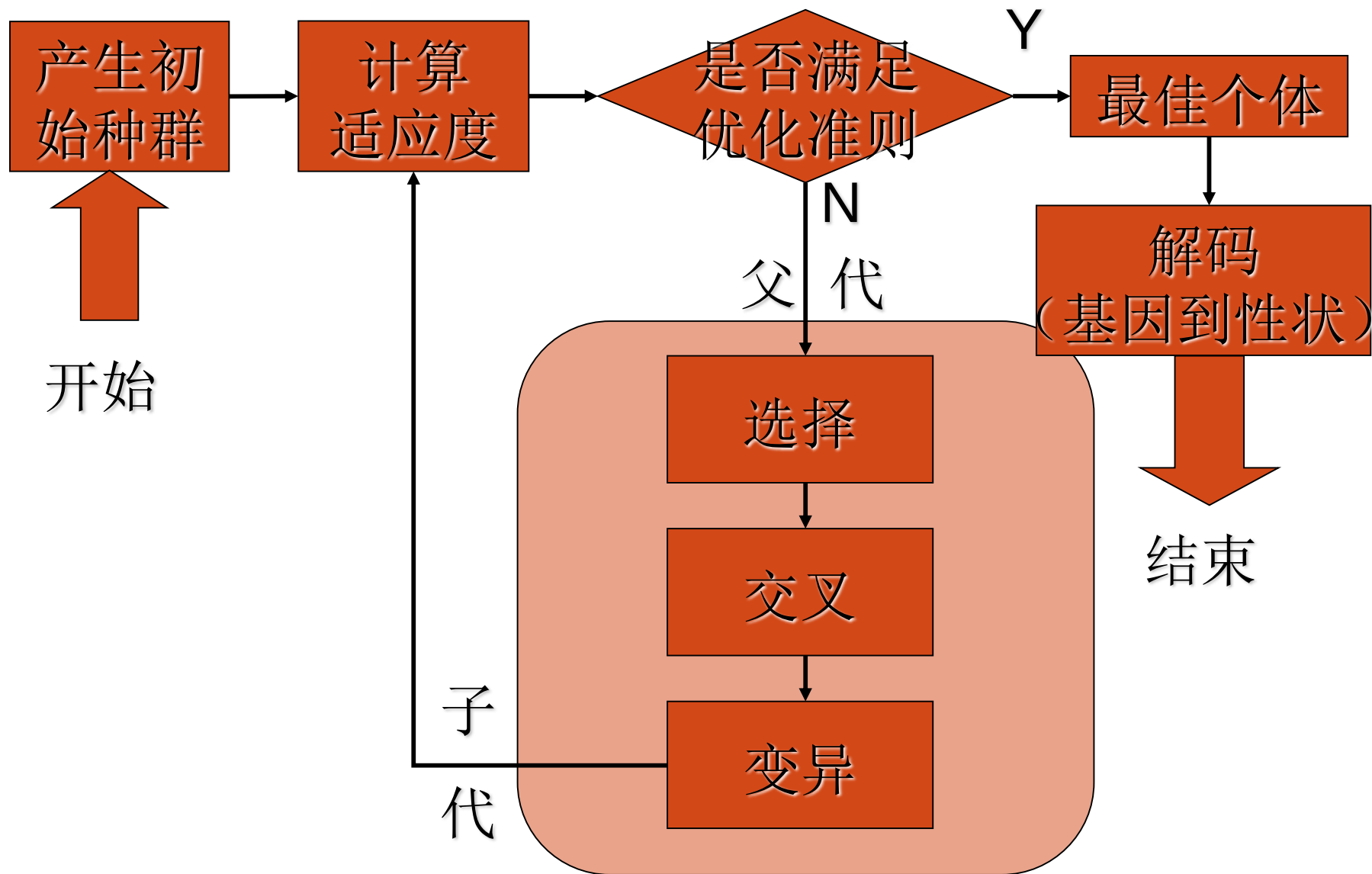


第四节：遗传算法流程

基本问题

- 用遗传算法求解问题需要解决以下五个问题

-
1. A **genetic representation** of potential solutions to the problem.(编码)
 2. A way to create a population (an **initial set of potential solutions**). (群体初始化)
 3. An **evaluation function** rating solutions in terms of their fitness.(个体评价)
 4. **Genetic operators** that alter the genetic composition of offspring (*selection, crossover, mutation*, etc.). (遗传算子)
 5. **Parameter values** that genetic algorithm uses (*population size, probabilities of applying genetic operators*, etc.). (参数选择).
-

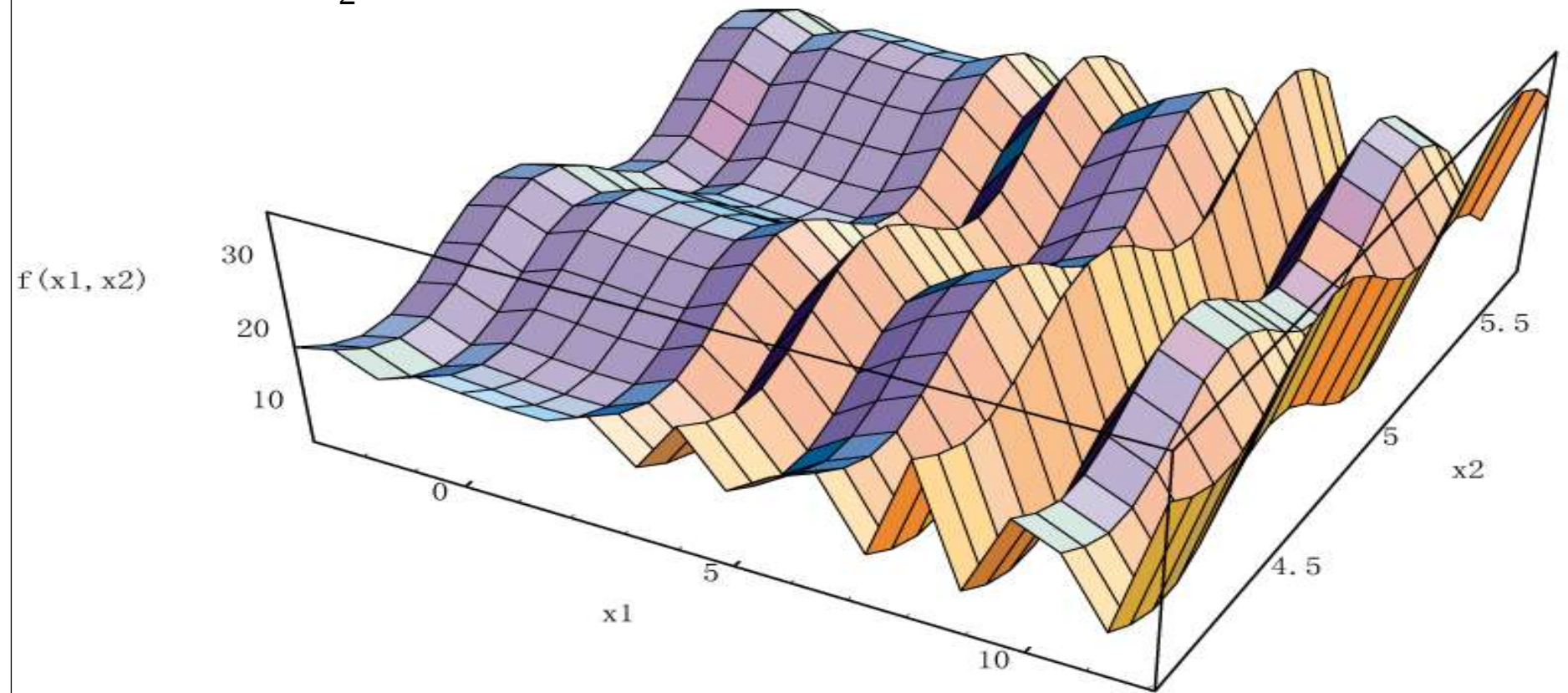


例子

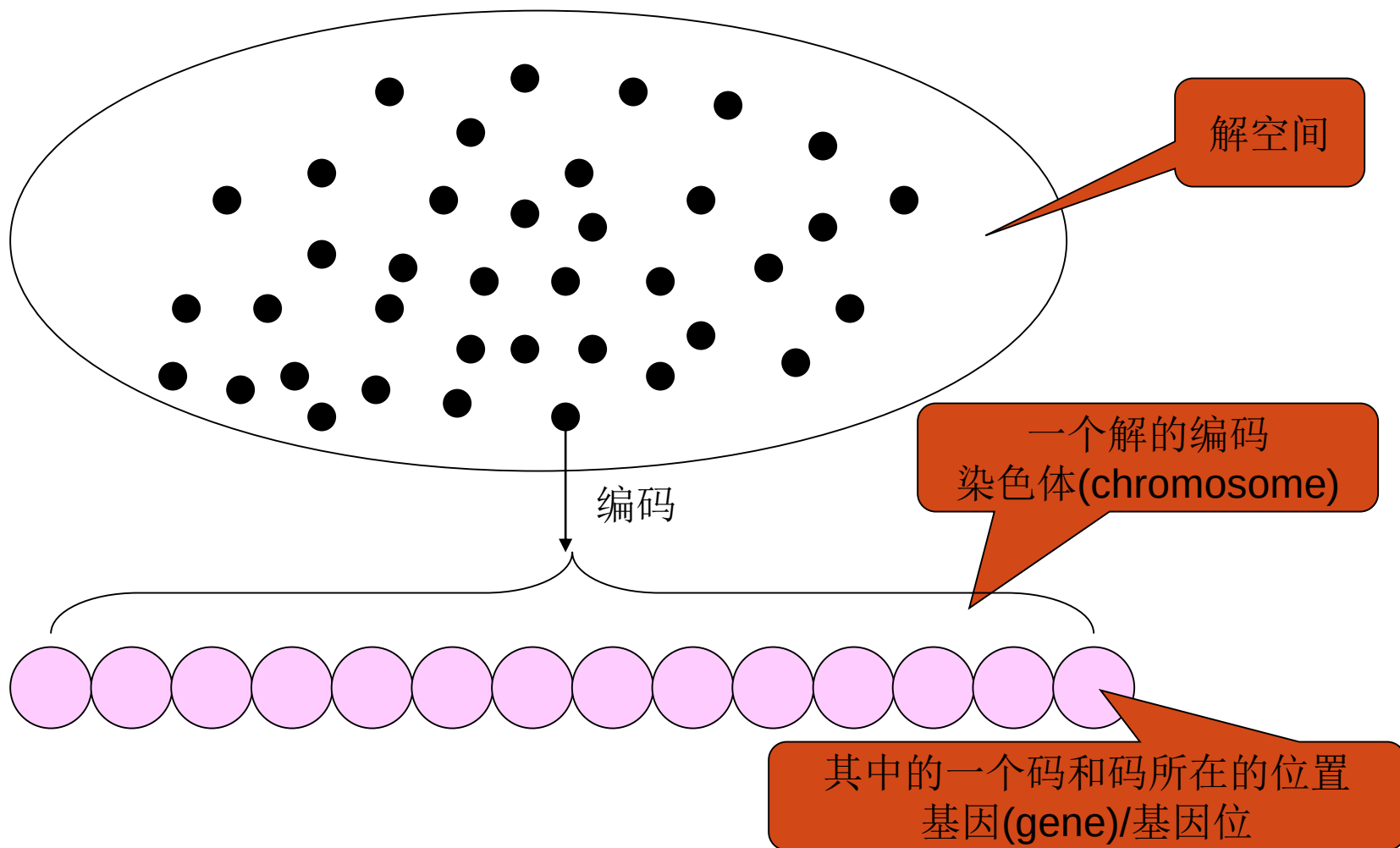
- We explain in detail about how a genetic algorithm actually works with a simple examples. (举例说明如何用遗传算法求解具体的优化问题)
- The numerical example of constrained optimization problem is given as follows:

$$\begin{aligned} \max \quad & f(x_1, x_2) = 21.5 + x_1 \cdot \sin(4\pi x_1) + x_2 \cdot \sin(20\pi x_2) \\ \text{s. t.} \quad & -3.0 \leq x_1 \leq 12.1 \\ & 4.1 \leq x_2 \leq 5.8 \end{aligned}$$

$$\begin{aligned} \max \quad & f(x_1, x_2) = 21.5 + x_1 \cdot \sin(4\pi x_1) + x_2 \cdot \sin(20\pi x_2) \\ \text{s. t.} \quad & -3.0 \leq x_1 \leq 12.1 \\ & 4.1 \leq x_2 \leq 5.8 \end{aligned}$$



编码问题



几个术语

- 基因型：

个体（染色体）

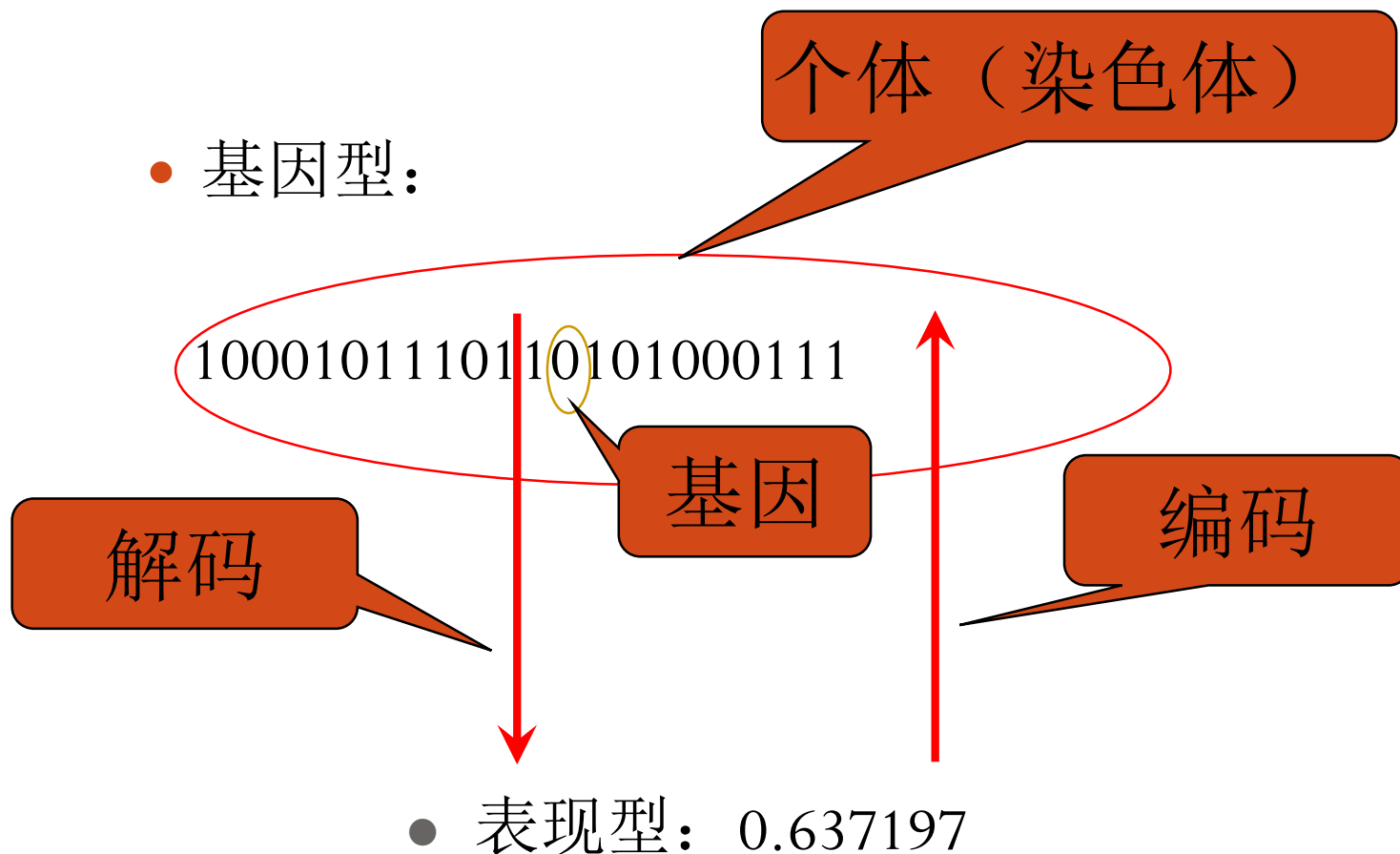
100010111011010101000111

基因

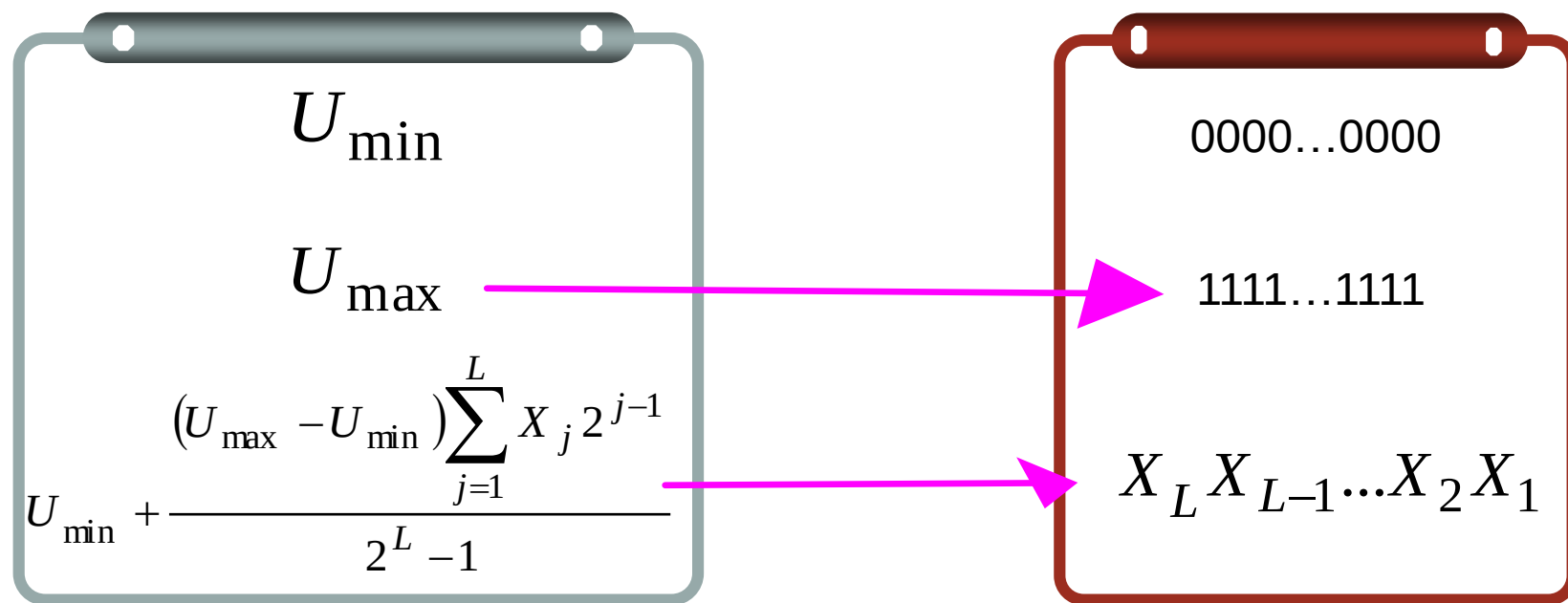
解码

编码

- 表现型：0.637197



二进制串表示



二进制串表示

- The domain of x_j is $[a_j, b_j]$ and the required precision is five places after the decimal point.(精度小数点后面四位)
- The precision requirement implies that the range of domain of each variable should be divided into at least $(b_j - a_j) \times 10^4$ size ranges.
- The required bits (denoted with m_j) for a variable is calculated as follows:

$$2^{m_j-1} < (b_j - a_j) \times 10^4 \leq 2^{m_j} - 1$$

- The mapping from a binary string to a real number for variable x_j is completed as follows:

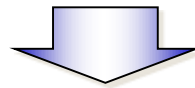
$$x_j = a_j + \text{decimal}(\text{substring}_j) \times \frac{b_j - a_j}{2^{m_j} - 1}$$

二进制串编码

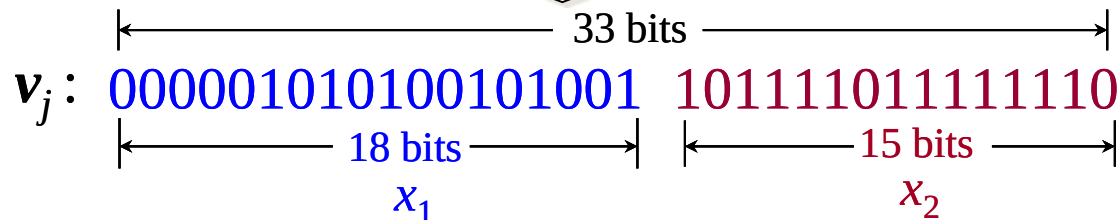
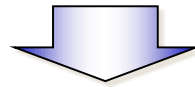
- The **precision requirement** implies that the range of domain of each variable should be divided into at least $(b_j - a_j) \times 10^4$ size ranges.
- The **required bits** (denoted with m_j) for a variable is calculated as follows:

$$x_1: (12.1 - (-3.0)) \times 10,000 = 151,000$$
$$2^{17} < 151,000 \leq 2^{18}, \quad m_1 = 18 \text{ bits}$$

$$x_2: (5.8 - 4.1) \times 10,000 = 17,000$$
$$2^{14} < 17,000 \leq 2^{15}, \quad m_2 = 15 \text{ bits}$$

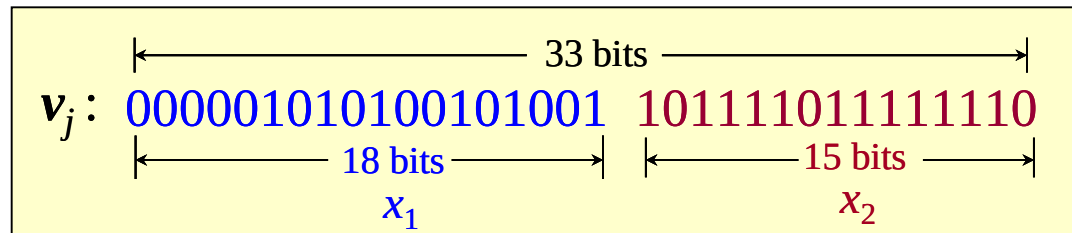


precision requirement: $m = m_1 + m_2 = 18 + 15 = 33$ bits



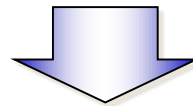
二进制串解码（基因型—>表现型）

- The mapping from a binary string to a real number for variable x_j is completed as follows:



	Binary Number	Decimal Number
x_1	000001010100101001	5417
x_2	101111011111110	24318

$$x_j = a_j + \text{decimal}(\text{substring}_j) \times \frac{b_j - a_j}{2^{m_j} - 1}$$



$$x_1 = -3.0 + 5417 \times \frac{12.1 - (-3.0)}{2^{18} - 1}$$

$$= -2.687069$$

$$x_2 = 4.1 + 24318 \times \frac{5.8 - 4.1}{2^{15} - 1}$$

$$= 5.361653$$

初始种群

$$\mathbf{v}_1 = [000001010100101001101111011111110] = [x_1 \ x_2] = [-2.687969 \ 5.361653]$$

$$\mathbf{v}_2 = [001110101110011000000010101001000] = [x_1 \ x_2] = [0.474101 \ 4.170144]$$

$$\mathbf{v}_3 = [111000111000001000010101001000110] = [x_1 \ x_2] = [10.419457 \ 4.661461]$$

$$\mathbf{v}_4 = [100110110100101101000000010111001] = [x_1 \ x_2] = [6.159951 \ 4.109598]$$

$$\mathbf{v}_5 = [000010111101100010001110001101000] = [x_1 \ x_2] = [-2.301286 \ 4.477282]$$

$$\mathbf{v}_6 = [1111101010110110000000010110011001] = [x_1 \ x_2] = [11.788084 \ 4.174346]$$

$$\mathbf{v}_7 = [110100010011111000100110011101101] = [x_1 \ x_2] = [9.342067 \ 5.121702]$$

$$\mathbf{v}_8 = [001011010100001100010110011001100] = [x_1 \ x_2] = [-0.330256 \ 4.694977]$$

$$\mathbf{v}_9 = [111110001011101100011101000111101] = [x_1 \ x_2] = [11.671267 \ 4.873501]$$

$$\mathbf{v}_{10} = [111101001110101010000010101101010] = [x_1 \ x_2] = [11.446273 \ 4.171908]$$

评价 (Evaluation)

- The process of **evaluating the fitness** of a chromosome consists of the following three steps:

input: chromosome $v_k, k=1, 2, \dots, popSize$

output: the fitness $eval(v_k)$

step 1: Convert the chromosome's genotype to its phenotype, *i.e.*, convert binary string into relative real values $x_k=(x_{k1}, x_{k2}), k = 1, 2, \dots, popSize$. (基因型到表现型)

step 2: Evaluate the objective function $f(x_k), k = 1, 2, \dots, popSize$.

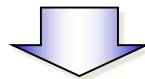
step 3: Convert the value of objective function into fitness. For the maximization problem, the fitness is simply equal to the value of objective function:

$$eval(v_k) = f(x_k), k = 1, 2, \dots, popSize.$$

$$eval(v_k) = f(x_i) \quad (k = 1, 2, \dots, popSize)$$

$$(i = 1, 2, \dots, n)$$

$$f(x_1, x_2) = 21.5 + x_1 \cdot \sin(4\pi x_1) + x_2 \cdot \sin(20\pi x_2)$$



Example: $(x_1=-2.687969, x_2= 5.361653)$

$$eval(v_1) = f(-2.687969, 5.361653) = 19.805119$$

评价 (Evaluation)

- An evaluation function plays **the role of the environment**, and it rates chromosomes in terms of **their fitness**.(适应度函数起到环境的作用)

- The fitness function values of above chromosomes are as follows:

$$eval(\mathbf{v}_1) = f(-2.687969, 5.361653) = 19.805119$$

$$eval(\mathbf{v}_2) = f(0.474101, 4.170144) = 17.370896$$

$$eval(\mathbf{v}_3) = f(10.419457, 4.661461) = 9.590546$$

$$eval(\mathbf{v}_4) = f(6.159951, 4.109598) = 29.406122$$

$$eval(\mathbf{v}_5) = f(-2.301286, 4.477282) = 15.686091$$

$$eval(\mathbf{v}_6) = f(11.788084, 4.174346) = 11.900541$$

$$eval(\mathbf{v}_7) = f(9.342067, 5.121702) = 17.958717$$

$$eval(\mathbf{v}_8) = f(-0.330256, 4.694977) = 19.763190$$

$$eval(\mathbf{v}_9) = f(11.671267, 4.873501) = 26.401669$$

$$eval(\mathbf{v}_{10}) = f(11.446273, 4.171908) = 10.252480$$

- It is clear that chromosome $\mathbf{v}_4$ is the **strongest** one and that chromosome $\mathbf{v}_3$ is the **weakest** one.

选择 (Selection 无偏见、平等)

- In most practices, a **roulette wheel approach** is adopted as the selection procedure, which is one of the fitness-proportional selection and can select a new population with respect to the probability distribution based on fitness values. (通过适应度来设计个体被选择的概率)
- The **roulette wheel** can be constructed with the following steps:

input: population $P(t-1)$, $C(t-1)$

output: population $P(t)$, $C(t)$

step 1: Calculate the total fitness for the population $F = \sum_{k=1}^{popSize} eval(\mathbf{v}_k)$

step 2: Calculate selection probability p_k for each chromosome $\mathbf{v}_k$

$$p_k = \frac{eval(\mathbf{v}_k)}{F}, \quad k = 1, 2, \dots, popSize$$

step 3: Calculate cumulative probability q_k for each chromosome $\mathbf{v}_k$

$$q_k = \sum_{j=1}^k p_j, \quad k = 1, 2, \dots, popSize$$

step 4: Generate a random number r from the range $[0, 1]$.

step 5: If $r \leq q_1$, then select the first chromosome $\mathbf{v}_1$; otherwise, select the k th chromosome $\mathbf{v}_k$ ($2 \leq k \leq popSize$) such that $q_{k-1} < r \leq q_k$.

- 选择的例子:

input: population $P(t-1)$, $C(t-1)$

output: population $P(t)$, $C(t)$

step 1: Calculate the total fitness F for the population.

$$F = \sum_{k=1}^{10} eval(\mathbf{v}_k) = 178.135372$$

step 2: Calculate selection probability p_k for each chromosome $\mathbf{v}_k$.

$$\begin{aligned} p_1 &= 0.111180, & p_2 &= 0.097515, & p_3 &= 0.053839, & p_4 &= 0.165077, \\ p_5 &= 0.088057, & p_6 &= 0.066806, & p_7 &= 0.100815, & p_8 &= 0.110945, \\ p_9 &= 0.148211, & p_{10} &= 0.057554 \end{aligned}$$

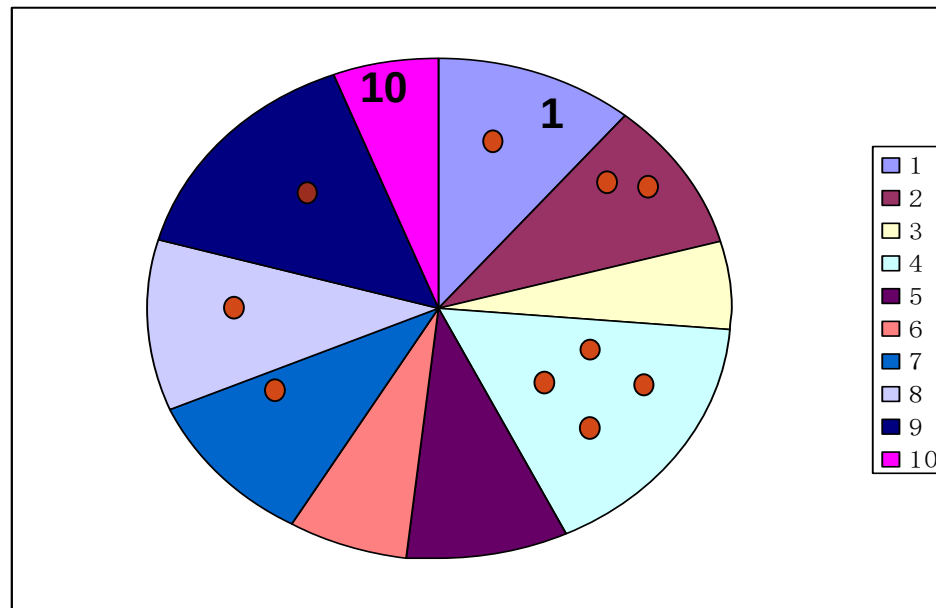
step 3: Calculate cumulative probability q_k for each chromosome $\mathbf{v}_k$.

$$\begin{aligned} q_1 &= 0.111180, & q_2 &= 0.208695, & q_3 &= 0.262534, & q_4 &= 0.427611, \\ q_5 &= 0.515668, & q_6 &= 0.582475, & q_7 &= 0.683290, & q_8 &= 0.794234, \\ q_9 &= 0.942446, & q_{10} &= 1.000000 \end{aligned}$$

step 4: Generate a random number r from the range $[0,1]$.

$$\begin{array}{ccccc} 0.301431, & 0.322062, & 0.766503, & 0.881893, & 0.350871, \\ 0.583392, & 0.177618, & 0.343242, & 0.032685, & 0.197577 \end{array}$$

$q_1 = 0.111180$, $q_2 = 0.208695$, $q_3 = 0.262534$, $q_4 = 0.427611$,
 $q_5 = 0.515668$, $q_6 = 0.582475$, $q_7 = 0.683290$, $q_8 = 0.794234$,
 $q_9 = 0.942446$, $q_{10} = 1.000000$



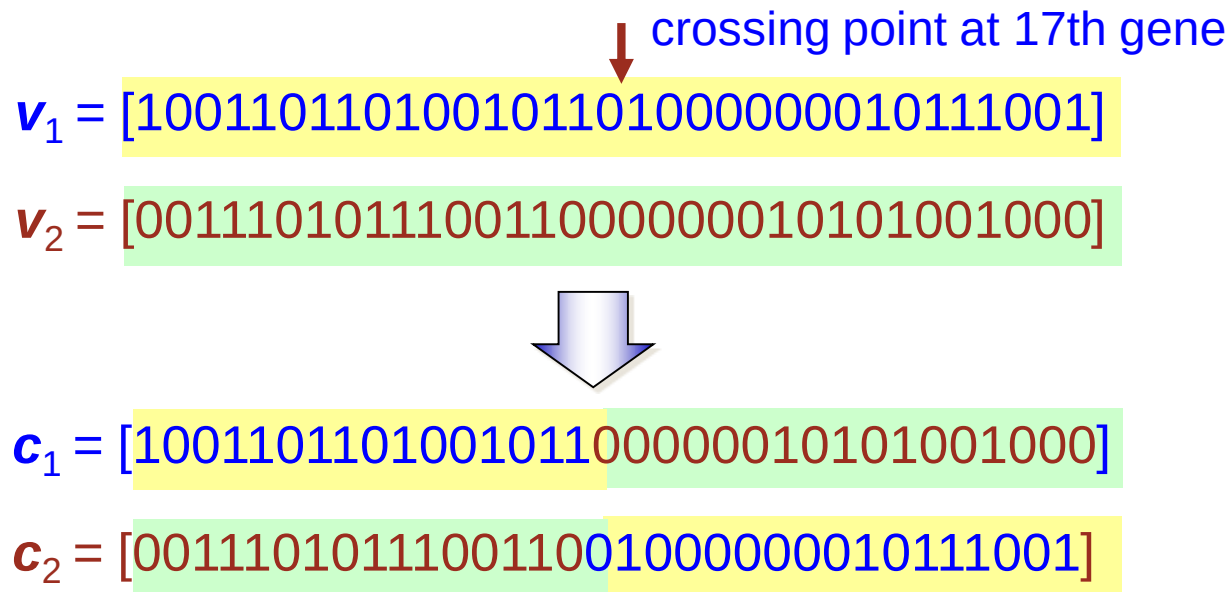
0.301431, 0.322062, 0.766503, 0.881893, 0.350871,
 0.583392, 0.177618, 0.343242, 0.032685, 0.197577

step 5: $q_3 < r_1 = 0.301432 \leq q_4$, it means that the chromosome $\mathbf{v}_4$ is selected for new population; $q_3 < r_2 = 0.322062 \leq q_4$, it means that the chromosome $\mathbf{v}_4$ is selected again, and so on. Finally, the new population consists of the following chromosome.

$$\begin{aligned}\mathbf{v}_1' &= [100110110100101101000000010111001] & (\mathbf{v}_4) \\ \mathbf{v}_2' &= [100110110100101101000000010111001] & (\mathbf{v}_4) \\ \mathbf{v}_3' &= [001011010100001100010110011001100] & (\mathbf{v}_8) \\ \mathbf{v}_4' &= [111110001011101100011101000111101] & (\mathbf{v}_9) \\ \mathbf{v}_5' &= [100110110100101101000000010111001] & (\mathbf{v}_4) \\ \mathbf{v}_6' &= [110100010011111000100110011101101] & (\mathbf{v}_7) \\ \mathbf{v}_7' &= [0011101011100110000000010101001000] & (\mathbf{v}_2) \\ \mathbf{v}_8' &= [100110110100101101000000010111001] & (\mathbf{v}_4) \\ \mathbf{v}_9' &= [000001010100101001101111011111110] & (\mathbf{v}_1) \\ \mathbf{v}_{10}' &= [0011101011100110000000010101001000] & (\mathbf{v}_2)\end{aligned}$$

交叉 (Crossover (One-cut point))

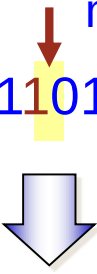
- Crossover used here is [one-cut point method](#), which random selects one cut point.
- **Exchanges** the right parts of two parents to generate offspring.
- Consider two chromosomes as follow and the cut point is randomly selected after the 17th gene:



变异 (Mutation)

- Alters one or more genes with a probability equal to the mutation rate. (改变一个或多个基因)
- Assume that the 16th gene of the chromosome v_1 is selected for a mutation.
- Since the gene is 1, it would be flipped into 0. So the chromosome after mutation would be:

mutating point at 16th gene



$v_1 = [100110110100101101000000010111001]$

$c_1 = [100110110100101000000010101001000]$

下一代 (Next Generation)

$$\begin{aligned} \mathbf{v}_1' &= [100110110100101101000000010111001], & f(6.159951, 4.109598) &= 29.406122 \\ \mathbf{v}_2' &= [100110110100101101000000010111001], & f(6.159951, 4.109598) &= 29.406122 \\ \mathbf{v}_3' &= [001011010100001100010110011001100], & f(-0.330256, 4.694977) &= 19.763190 \\ \mathbf{v}_4' &= [111110001011101100011101000111101], & f(11.907206, 4.873501) &= 5.702781 \\ \mathbf{v}_5' &= [100110110100101101000000010111001], & f(8.024130, 4.170248) &= 19.91025 \\ \mathbf{v}_6' &= [110100010011111000100110011101101], & f(9.34067, 5.121702) &= 17.958717 \\ \mathbf{v}_7' &= [100110110100101101000000010111001], & f(6.159951, 4.109598) &= 29.406122 \\ \mathbf{v}_8' &= [100110110100101101000000010111001], & f(6.159951, 4.109598) &= 29.406122 \\ \mathbf{v}_9' &= [000001010100101001101111011111110], & f(-2.687969, 5.361653) &= 19.805199 \\ \mathbf{v}_{10}' &= [0011101011100110000000010101001000], & f(0.474101, 4.170248) &= 17.370896 \end{aligned}$$

最终结果

- The test run is terminated after 1000 generations.
- We obtained the best chromosome in the 884th generation as follows:

$$\max \quad f(x_1, x_2) = 21.5 + x_1 \cdot \sin(4\pi x_1) + x_2 \cdot \sin(20\pi x_2)$$

$$\text{s. t.} \quad -3.0 \leq x_1 \leq 12.1$$

$$4.1 \leq x_2 \leq 5.8$$

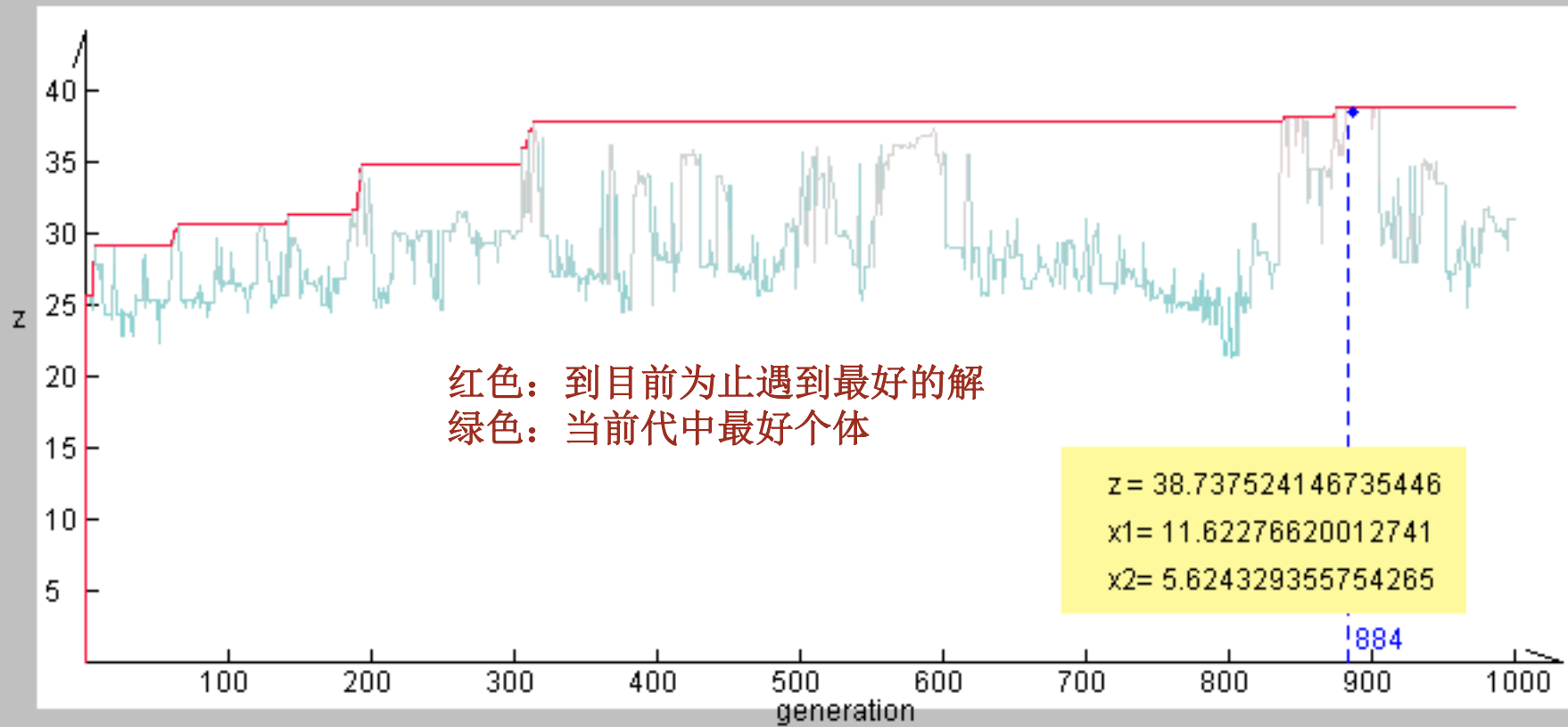
$$\begin{aligned} eval(\mathbf{v}^*) &= f(11.622766, 5.624329) \\ &= 38.737524 \end{aligned}$$

$$x_1^* = 11.622766$$

$$x_2^* = 5.624329$$

$$f(x_1^*, x_2^*) = 38.737524$$

$\text{maxGen: } 1000$ $p_C: 0.25$ $p_M: 0.01$

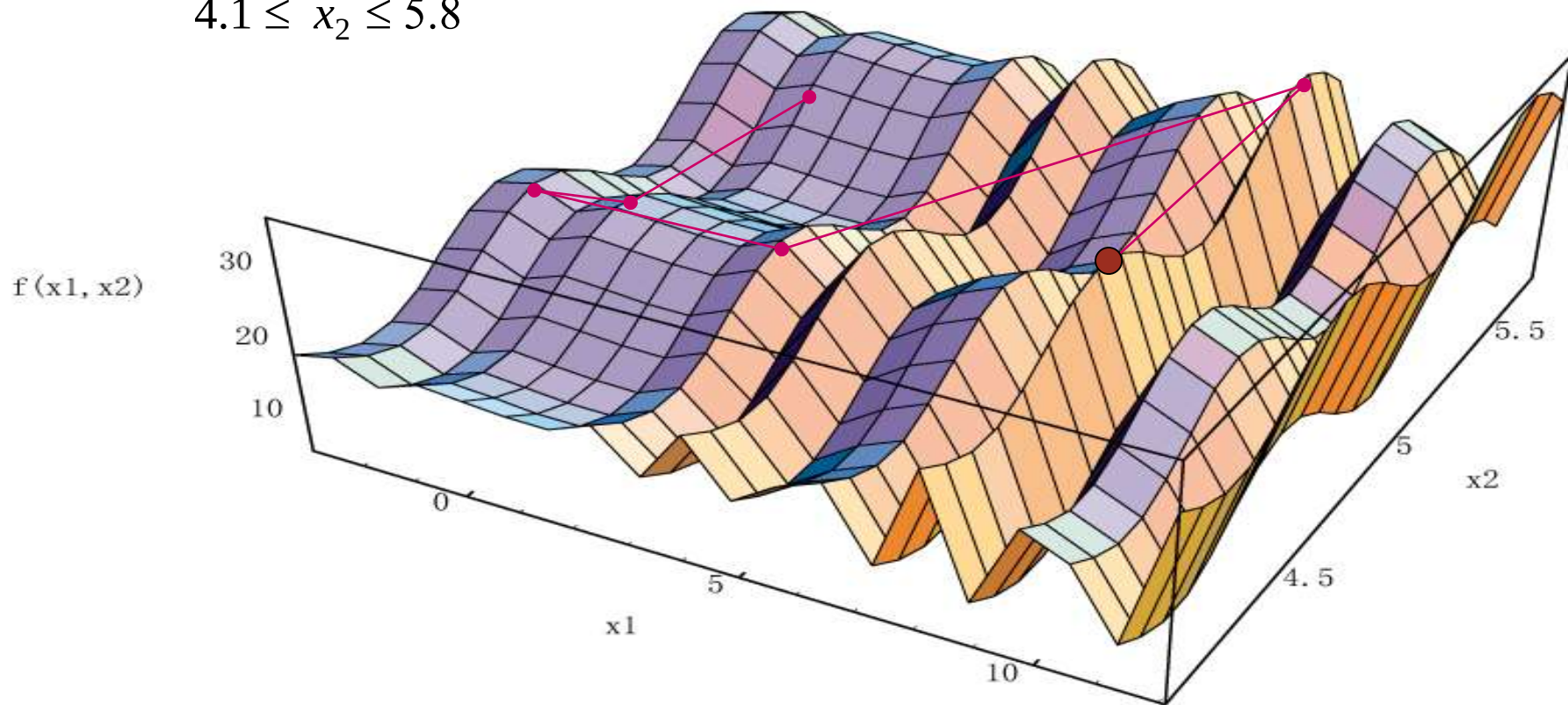


• 演化过程

$$\max \quad f(x_1, x_2) = 21.5 + x_1 \cdot \sin(4\pi x_1) + x_2 \cdot \sin(20\pi x_2)$$

$$\text{s. t.} \quad -3.0 \leq x_1 \leq 12.1$$

$$4.1 \leq x_2 \leq 5.8$$

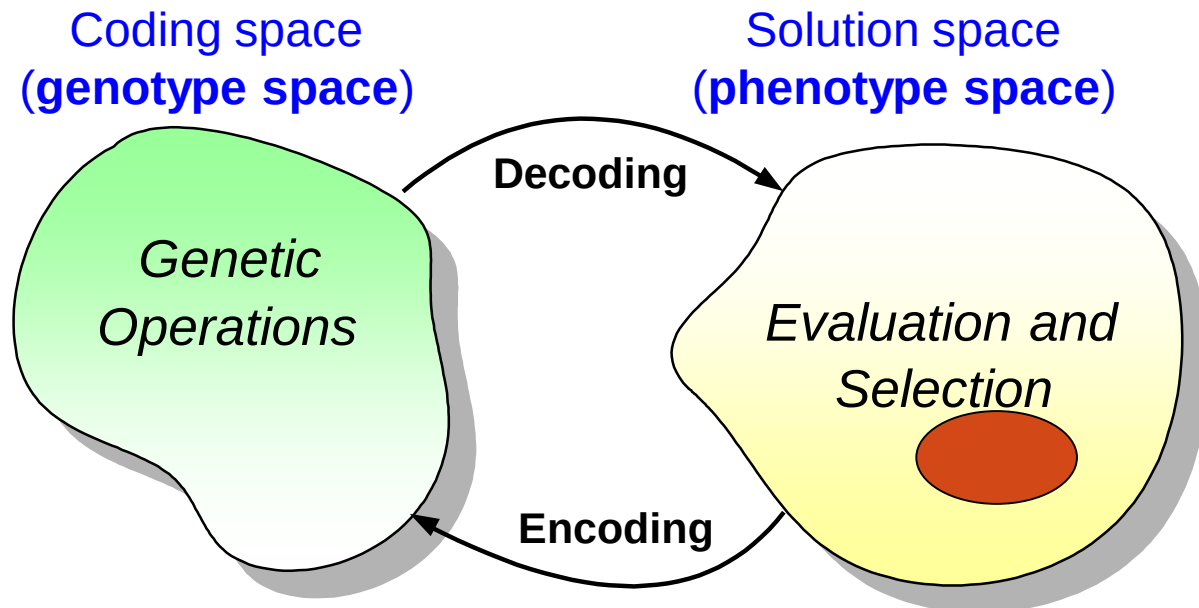


第五节：遗传算法构成模块

模块一： 编码

- 遗传算法通过遗传操作对群体中具有某种结构形式的**个体处理**，从而生成新的群体，逐渐逼近最优解。它不直接处理问题空间的决策变量，必须转换成由基因按一定结构组成的染色体。
- 该转换为编码，编码过程是问题空间向编码空间的映射过程。编码方法除了决定了个体的染色体排列形式之外，它还决定了个体的解码方法；编码方法也影响到交叉算子、变异算子等遗传算子的运算方法。

- Basic features of genetic algorithms is that they work on **coding space** and **solution space** alternatively:
 - **Genetic operations** work on **coding space** (chromosomes)
 - While **evaluation and selection** work on **solution space**.
 - Natural selection is the link between chromosomes and the performance of their decoded solutions.(通过自然选择将染色体和解连接起来)



- There are three critical issues emerged concerning with the encoding and decoding between chromosomes and solutions (or the mapping between **phenotype** and **genotype**): 需要考虑下面三个问题
-

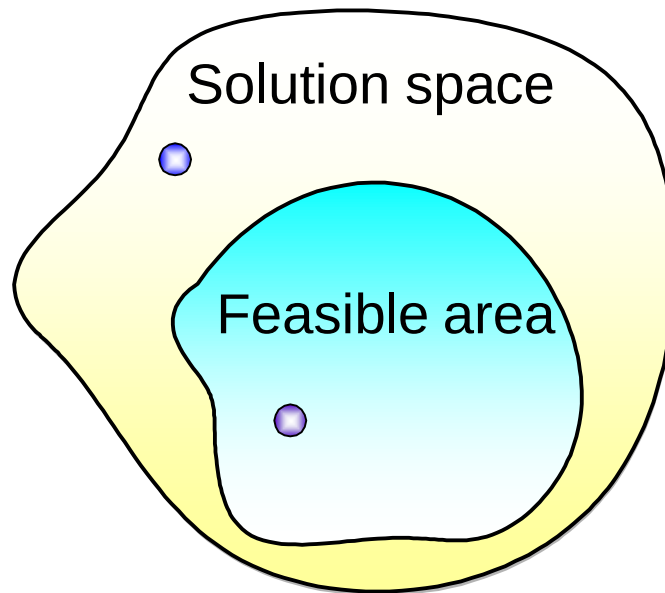
- The **feasibility** of a chromosome(染色体的可行性, 是否在可行域内)
 - The **feasibility** refers to the phenomenon that whether or not a solution decoded from a chromosome lies in the feasible region of a given problem.
 - The **legality** of a chromosome(染色体的合法性, 是否为问题的一个解)
 - The **legality** refers to the phenomenon that whether or not a chromosome represents a solution to a given problem.
 - The **uniqueness** of mapping(映射的唯一性)
-

COP : 约束优化问题 (非可行解的产生)

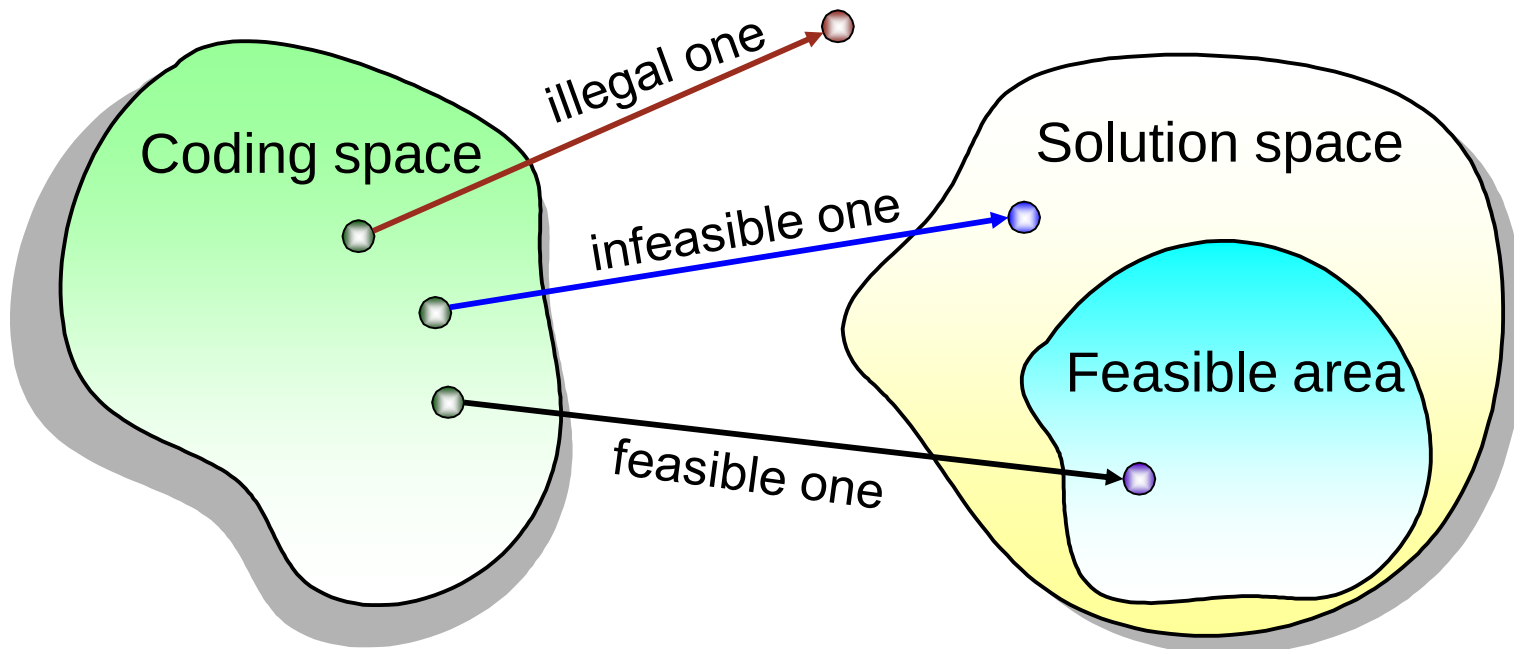
$\max f(x), x \in D$ (D称为解空间)

st $g(x) < 0, h(x) = 0$

可行域 $C = \{x | x \in D, g(x) < 0, h(x) = 0\}$



- Feasibility and Legality as follows



Feasibility and Legality

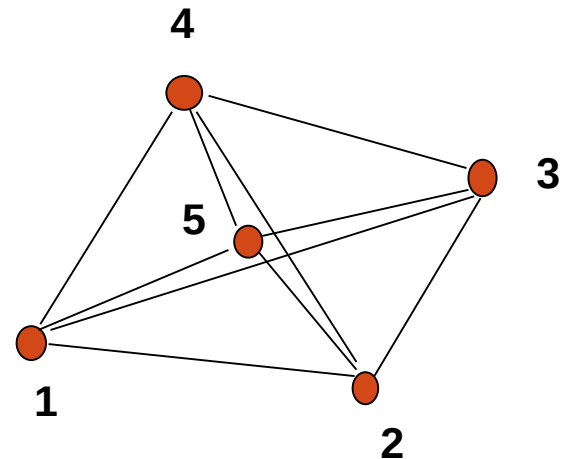
例子：TSP问题的非法解和唯一性

采用整数编码： $a\ b\ c\ d\ e$ ，其中
 $a,b,c,d,e \in \{1,2,3,4,5\}$

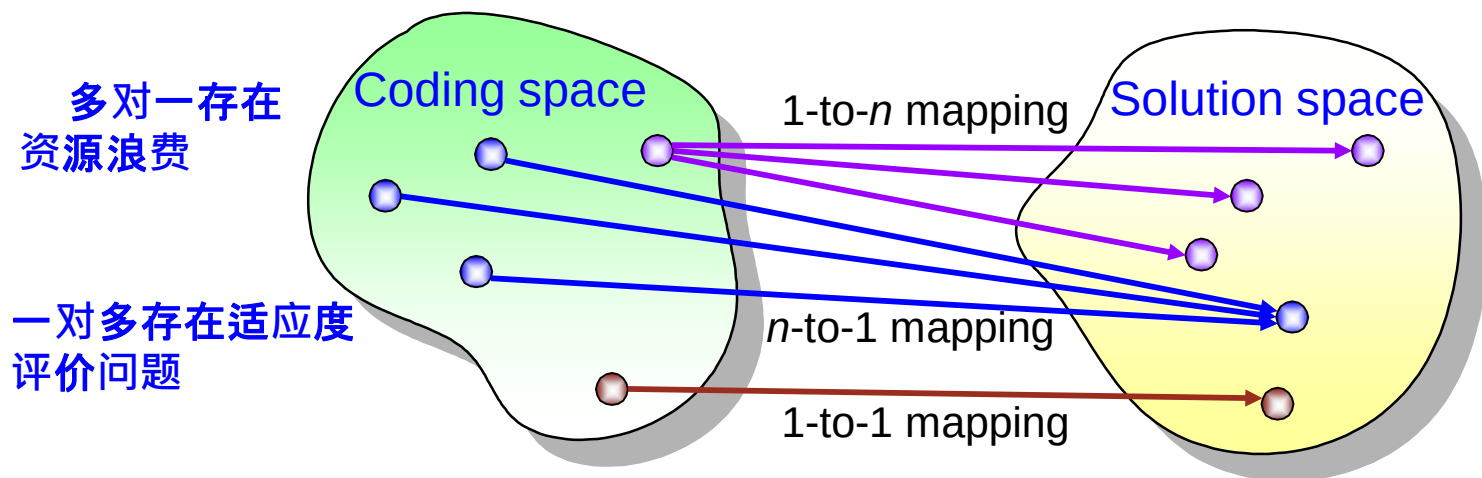
3 2 1 4 5 $3 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 5 \rightarrow 3$

显然 **2 3 5 2 1** 属于码空间但不是合法的解

同时 **3 2 1 4 5** 与 **1 4 5 3 2** 对应同一条路径



- The **mapping from chromosomes to solutions** (decoding) may belong to one of the following three cases:
 - 1-to-1 mapping
 - n -to-1 mapping
 - 1-to- n mapping
- The **1-to-1 mapping** is the best(最好) one among three cases and **1-to- n mapping** is the most undesired one(最差).
 - We need to consider these problems carefully when designing a new nonstring coding so as to build an effective genetic algorithm.



编码性能评价

- 不冗余.码空间到解空间是1对1影射.
- 合法性.对编码的任意排列对对应一个解.
- 完备性.任意一个解都对应一个排列.
- 因果性：有变异带来基因空间中小的扰动，在表现型空间也对应较小的扰动！

- 如何将问题的候选解用染色体来表示是遗传算法关键
 - In **Holland's work**, encoding is carried out using **binary strings**.
 - 二进制编码存在悬崖问题：表现型空间中距离很小的个体在基因型空间的Hamming距离可能很大，为了翻越这个悬崖，个体的所有位都需要改变，而实际算子交叉和变异翻越悬崖的可能性却较小。

二进制编码与格雷编码

	二进制编码	格雷编码
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

$$g_1 = b_1$$

$$g_l = b_{l-1} \bar{b}_l + \bar{b}_{l-1} b_l$$

式中， b_l 是二进制编码 $b_1 b_2 \cdots b_n$ 的第 l 位比特， b_1 为最重要的一位； $\bar{b}_l$ 表示非 b_l ， $+$ 表示逻辑或，乘表示逻辑与。

模块二：种群规模设定

- 从群体多样性方面考虑，群体越大越好，避免陷入局部最优。
- 从计算效率，群体规模应小。群体越大，其适应度评估次数增加，计算量增加。

- 另一方面，群体规模太小，会使遗传算法的搜索空间分布范围有限，因而搜索有可能停止在未成熟阶段，引起未成熟收敛(premature convergence)现象。显然，要避免未成熟收敛现象，必须保持群体的多样性，群体规模不能太小。
- 应该针对不同的实际问题，确定不同的种群规模。Goldberg证明了在二进制编码的前提下，若个体长度为 L ，则种群规模的最优值为 $2^{L/2}$

初始化

产生初始种群的方法通常有两种：

- 完全随机的方法产生的，它适合于对问题的解无任何先验知识的情况；
- 根据某些先验知识转变为必须满足的一组要求，然后在满足这些要求的解中再随机地选取样本。

模块三：适应度函数(Fitness Function)

- 适应度函数(Fitness Function)的选取直接影响到遗传算法的收敛速度以及能否找到最优解，因为在进化搜索中基本不利用外部信息，仅以适应度函数为依据，利用种群每个个体的适应度来指导搜索。
- 在实际问题中，适应度函数与问题的目标函数是不完全一致的，如有的问题的目标是要要求的最小值(费用问题)，而有的问题的目标是要求得最大值(利润函数)。

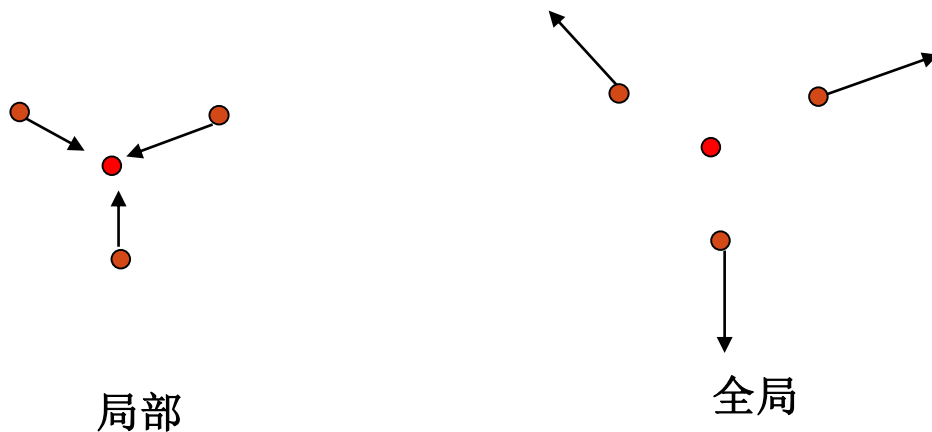
模块四：遗传算子

- 当很难事先获得问题求解步骤时，搜索成为有效的求解方法。代表搜索方法：局部搜索和随机搜索。
- 经典算法的局部搜索能力较强，遗传算法是一种结合了局部搜索和随机两种能力的通用搜索方法，对深度搜索和广度搜索进行平衡！

Exploration(全局): Discovering promising areas in the search space, i.e. gaining information on the problem(全局),crossover(算子)

Exploitation（局部）: Optimising within a promising area, i.e. using information(局部),mutation(算子)

Exploration与Exploitation



搜索 $\mathbf{x}' = \mathbf{x} + \mathbf{d}$

局部 $|\mathbf{x}' - \mathbf{x}| = \mathbf{d}$ 较小，结束时，提高精度

全局 $|\mathbf{x}' - \mathbf{x}| = \mathbf{d}$ 较大，遗传算法开始阶段，扩大搜索范围

Crossover and Mutation

- Genetic operators are used to alter the genetic composition of chromosomes
- There are two common genetic operators:
 - **Crossover**
 - Operating on two chromosomes at a time and generating offspring by combining both chromosomes' features
 - **Mutation**
 - Producing random changes in various chromosomes.

Conventional Operators

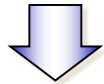
One-cut Point Crossover:

crossing point at k th position

parents

$$\mathbf{x} = [x_1, x_2, \dots, x_k, x_{k+1}, x_{k+2}, \dots, x_n]$$

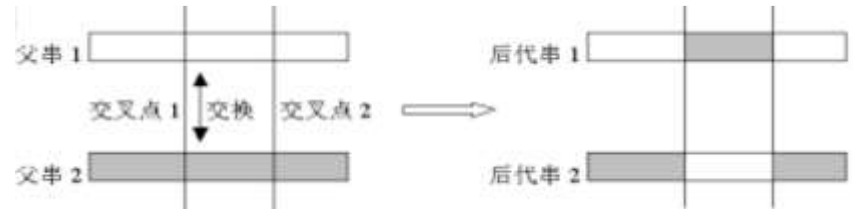
$$\mathbf{y} = [y_1, y_2, \dots, y_k, y_{k+1}, y_{k+2}, \dots, y_n]$$



offspring

$$\mathbf{x}' = [x_1, x_2, \dots, x_k, y_{k+1}, y_{k+2}, \dots, y_n]$$

$$\mathbf{y}' = [y_1, y_2, \dots, y_k, x_{k+1}, x_{k+2}, \dots, x_n]$$



- TSP交叉操作算子

$$p1 = 2\ 4\ 3\ |\ 1\ 8\ 6\ 7\ |\ 5\ 9, \quad p2 = 2\ 1\ 3\ |\ 4\ 5\ 6\ 7\ |\ 8\ 9$$

- 然后移走 $p1$ 中已在 $p2$ 中的城市4、5、6和7后，得到

$$2-3-1-8-9$$

- 该序列顺次放在 $o1$ 中：

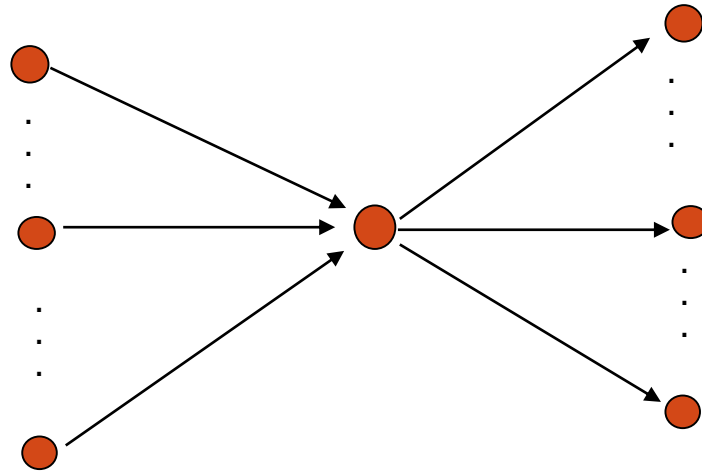
$$o1 = (2\ 3\ 1\ |\ 4\ 5\ 6\ 7\ |\ 8\ 9)$$

- 类似地，可以得到另一个后代：

$$o2 = (2\ 3\ 4\ |\ 1\ 8\ 6\ 7\ |\ 5\ 9)$$

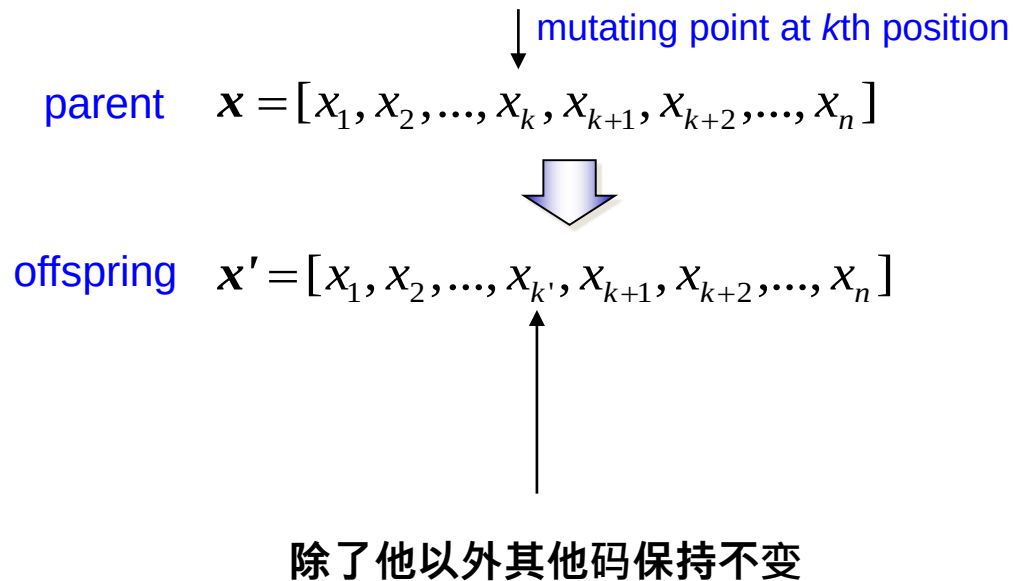
多父辈交叉算子

- 多父辈交叉有利于提高遗传算法的性能的随着在交叉操作中多父辈的引入，降低了超级个体将自身复制到子代中的可能性，这就意味着带来了更为多样的解空间搜索结果，从而减少了遗传算法早熟的危险.



Random Mutation

□ Random Mutation



Insert Mutation

- Pick two allele values at random (选择两个等位基因)
- Move the second to follow the first, shifting the rest along to accommodate (将第二个插入到第一个之后)
- Note that this preserves most of the order and the adjacency information (优点保护大部分基因位信息的临近关系)

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---



1	2	5	3	4	6	7	8	9
---	---	---	---	---	---	---	---	---

Swap mutation (交换)

Pick two alleles at random and swap their positions
Preserves most of adjacency information (4 links broken),
disrupts order more

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

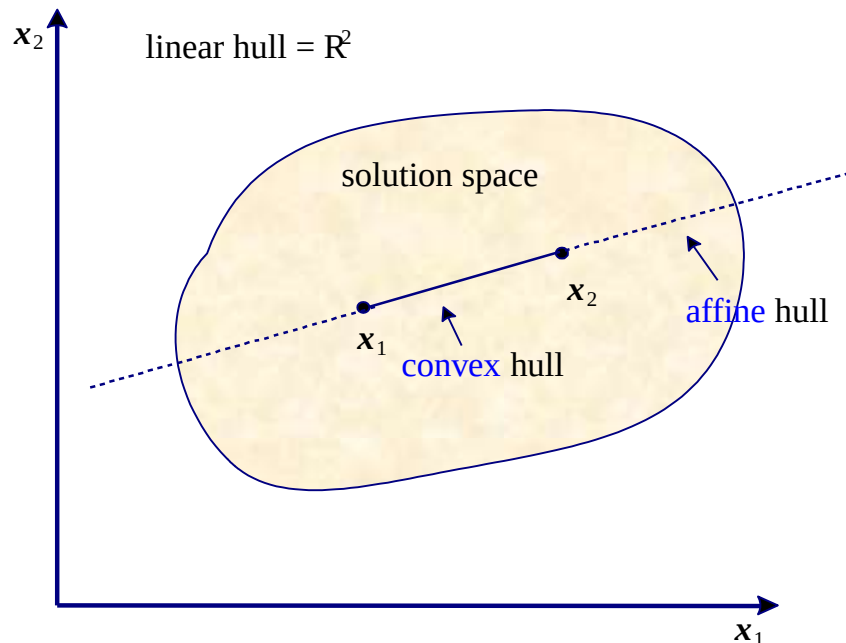


1	5	3	4	2	6	7	8	9
---	---	---	---	---	---	---	---	---

Arithmetical Operators

• Crossover

- Suppose that these are two parents x_1 and x_2 , the offspring can be obtained by $\lambda_1 x_1 + \lambda_2 x_2$ with different multipliers λ_1 and λ_2 .



$$\mathbf{x}_1' = \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2$$

$$\mathbf{x}_2' = \lambda_1 \mathbf{x}_2 + \lambda_2 \mathbf{x}_1$$

- Convex Crossover

If $\lambda_1 + \lambda_2 = 1$, $\lambda_1 > 0$, $\lambda_2 > 0$

- Affine Crossover 仿射交叉

If $\lambda_1 + \lambda_2 = 1$

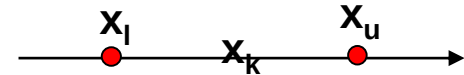
- Linear Crossover

If $\lambda_1 + \lambda_2 \leq 2$, $\lambda_1 > 0$, $\lambda_2 > 0$

Fig 1.2 Illustration showing convex, affine, and linear hull

Arithmetical Operators(实数变异)

- Nonuniform Mutation (Dynamic Mutation)



- For a given parent x , if the element x_k of it is selected for mutation, the resulting offspring is $x' = [x_1 \dots x_{k'} \dots x_n]$,

where $x_{k'}$ is randomly selected from two possible choice: t 为代数

$$x'_k = x_k + \Delta(t, x_k^U - x_k) \quad \text{or} \quad x'_k = x_k - \Delta(t, x_k - x_k^L)$$

- where x_k^U and x_k^L are the upper and lower bounds for x_k .
- The function $\Delta(t, y)$ returns a value in the range $[0, y]$ such that the value of $\Delta(t, y)$ approaches to 0 as t increases (t is the generation number):

$$\Delta(t, y) = y \cdot r \cdot \left(1 - \frac{t}{T}\right)^b$$

where r is a random number from $[0, 1]$, T is the maximal generation number, and b is a parameter determining the degree of nonuniformity.

Direction-based Operators

- This operation use the values of objective function in **determining the direction of genetic search**:(通过目标函数值来决定搜索的方向)
- **Direction-based crossover**
 - Generate a single offspring x' from two parents x_1 and x_2 according to the following rules:

$$\mathbf{x}' = r \cdot (\mathbf{x}_2 - \mathbf{x}_1) + \mathbf{x}_2$$

where $0 < r \leq 1$. (x_2 不差于 x_1)

- **Directional mutation**

- The offspring after mutation would be:

$$\mathbf{x}' = \mathbf{x} + r \cdot \mathbf{d}$$

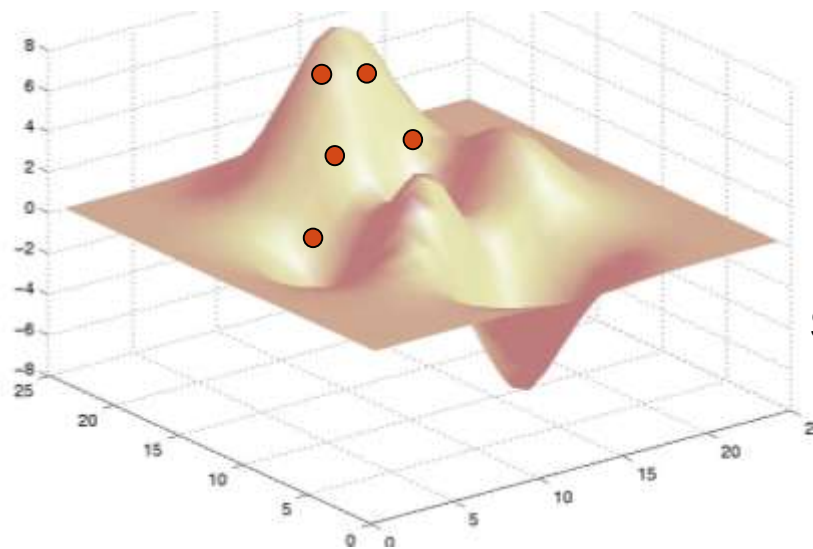
$$\text{where } \mathbf{d} = \frac{f(x_1, \dots, x_i + \Delta x_i, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{\Delta x_i}$$

r = a random nonnegative real number

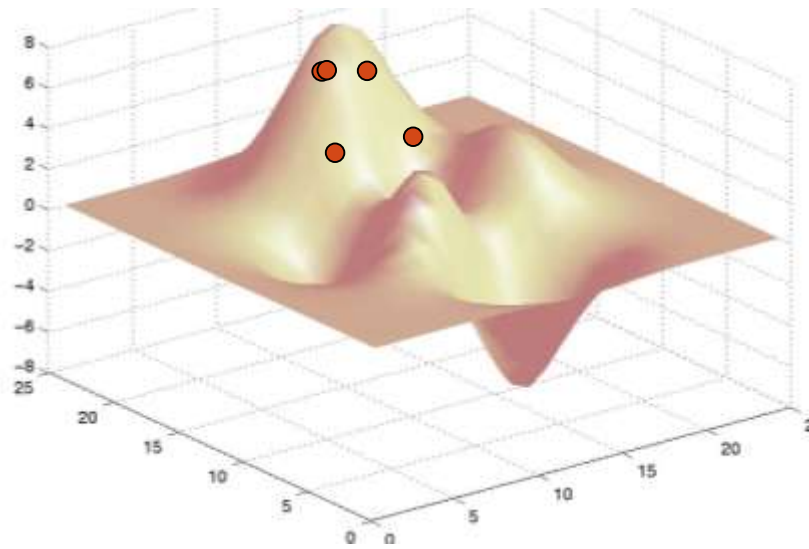
Selection

- The principle behind genetic algorithms is essentially Darwinian natural selection. (选择的基础是达尔文的适者生存理论)
- Selection provides the driving force in a genetic algorithm and the selection pressure is a critical in it. (合适的选择压力很重要)
 - Too much, the search will terminate prematurely
 - Too little, progress will be slower than necessary.
 - Low selection pressure is indicated at the start to the GA search in favor of a wide exploration of the search space.
 - High selection pressure is recommended at the end in order to exploit the most promising regions of the search space.
- The selection directs GA search towards promising regions in the search space.

选择的作用

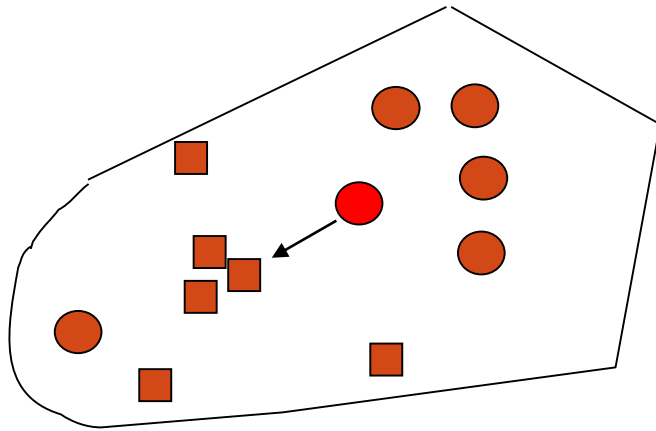


→
Selection



- In Holland's original GA, parents are replaced by their offspring soon after they give birth(生成后即替换).
 - This is called as generational replacement.(世代替换)
- Because genetic operations are blind in nature, offspring may be worse than their parents. (遗传算子属于盲搜索或随机搜索, 存在退化,所以上述算子不合适)
- De Jong proposed a crowding strategy(拥挤策略).
 - In the crowding model, when an offspring was born, one parent was selected to die. The dying parent was chosen as that parent was most closely resembled the new offspring using a simple bit-by-bit similarity count to measure resemblance.(拥挤原则, 当一个新个体产生, 死亡的个体应该是和他最接近的父代个体)

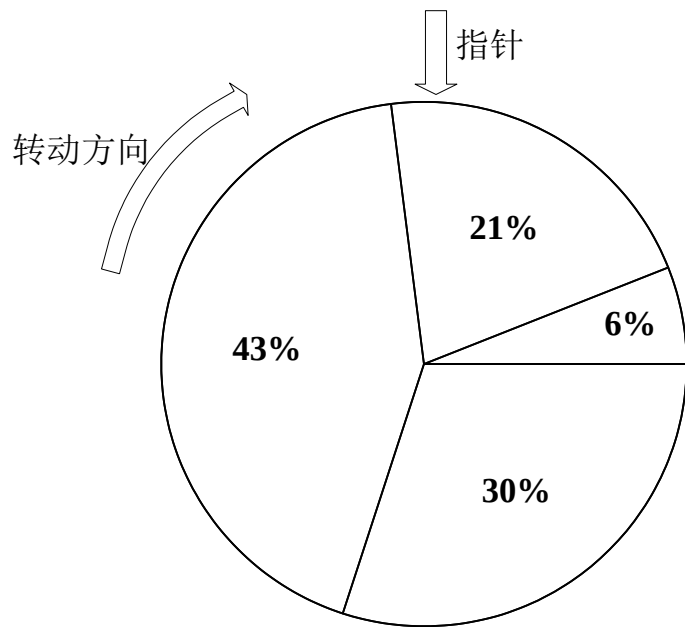
Selection(crowding strategy)



- 子代个体
- 父代个体
- 新生成的子代个体

个体之间的距离：码空间的距离
解空间的距离

轮盘赌选择（随机性选择）



/* once of roulette wheel selection

* 输出参数:

* 选中的染色体

*/

procedure RWS

1 $m \leftarrow 0;$

2 $r \leftarrow \text{Random}(0,1);$ //0至1的随机数

3 **for** $i=1$ to N

4 $m \leftarrow m + P_i;$

5 **if** $r \leq m$

6 **return** $i;$

7 **end if**

8 **end for**

end procedure

- **Deterministic Sampling(确定性选择)**

所谓确定性选择就是从父代和子代个体中选择最优的个体。

- **Deterministic procedures which select the best chromosomes from parents and offspring.**

Truncation selection(截断选择)

- $(\mu + \lambda)$: selection(μ 个父代, λ 个子代, 从 $\mu + \lambda$ 选择最好的 μ)
- (μ, λ) : selection(μ 个父代, λ 个子代, 从 λ 选择最好的 μ)
- Elitist selection(精英选择): 从父代中选择最好的

Goldberg 指出, GAs 中几种流行的选择机制中, $(\mu + \lambda)$ 选择机制具有最强的选择压。 $(\mu + \lambda)$ 选择允许 μ 个父代个体及其产生的 λ 个子代个体共同竞争, 选择 μ 个高适应值个体进入下一代, 仿真结果表明, 当在整个种群中进行随机交叉操作时, 用 $(\mu + \lambda)$ 选择能产生最快的局部收敛速度, 局部收敛时, 种群的平均适应值等于最优个体的适应值, 这表明, 整个种群中的个体适应值全部一样, 种群多样性损失殆尽。

- **Mixed Sampling** (混合选择同时具有随机性和确定性)
 - Contains both random and deterministic features simultaneously.
 - Tournament selection(竞赛选择): 取出一定数量个体成为竞赛规模, 选择其中最好的进入子代, 重复操作直到新的种群规模达到原来种群的规模
 - Binary tournament selection (规模为2的竞赛选择)
 - Stochastic tournament selection

选择比例存在问题

- 在算法的初期个别超级染色体具有绝对的优势控制整个选择过程，竞争过于强烈；
- 而在算法的晚期（大部分个体已经收敛）个体之间的适应度差别不大，竞争太弱，呈现随机搜索行为。
- 适应度变换机制可以解决这些问题。

- Selection Probability
 - **Fitness scaling** has a twofold intention:(适应度变换有两个目的)
 - To maintain a reasonable differential between relative fitness ratings of chromosomes.(维持个体之间的合理差距，加速竞争)
 - To prevent a too-rapid takeover by some supper chromosomes in order to meet the requirement to limit competition early on, but to stimulate it later.(避免个体之间的差距过大，限制竞争)
 - Suppose that the raw(原始) fitness f_k (e.g. objective function value) for the k -th chromosomes, **the scaled fitness f'_k** is:
$$f'_k = g(f_k)$$
 - Function $g(\cdot)$ may take different form to yield different scaling methods.

□ Scaling Mechanisms

■ Linear scaling

$$f'_k = a \times f_k + b$$

■ Power law scaling

$$f'_k = f_k^\alpha$$

■ Normalizing scaling

$$f'_k = \frac{f_k - f_{\min} + \gamma}{f_{\max} - f_{\min} + \gamma}$$

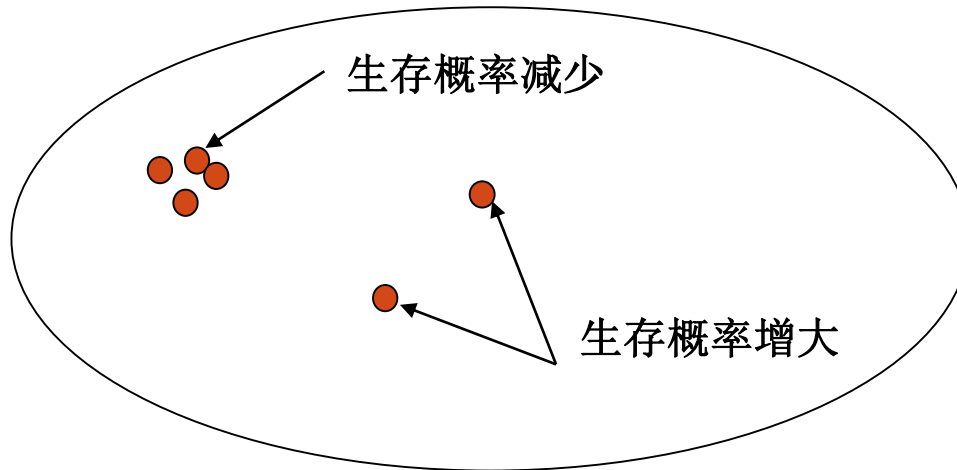
$0 < \gamma < 1$ (for maximization problem)

■ Boltzmann scaling

$$f'_k = e^{f_k/T}$$

共享方法

- 共享函数法。根据个体某个距离内与其他个体的临近程度来确定该个体的适应度应改变多少。在拥挤的峰周围的个体的复制概率受到抑制，利于其他个体产生后代。



第六节：发展

并行遗传算法

- 并行计算

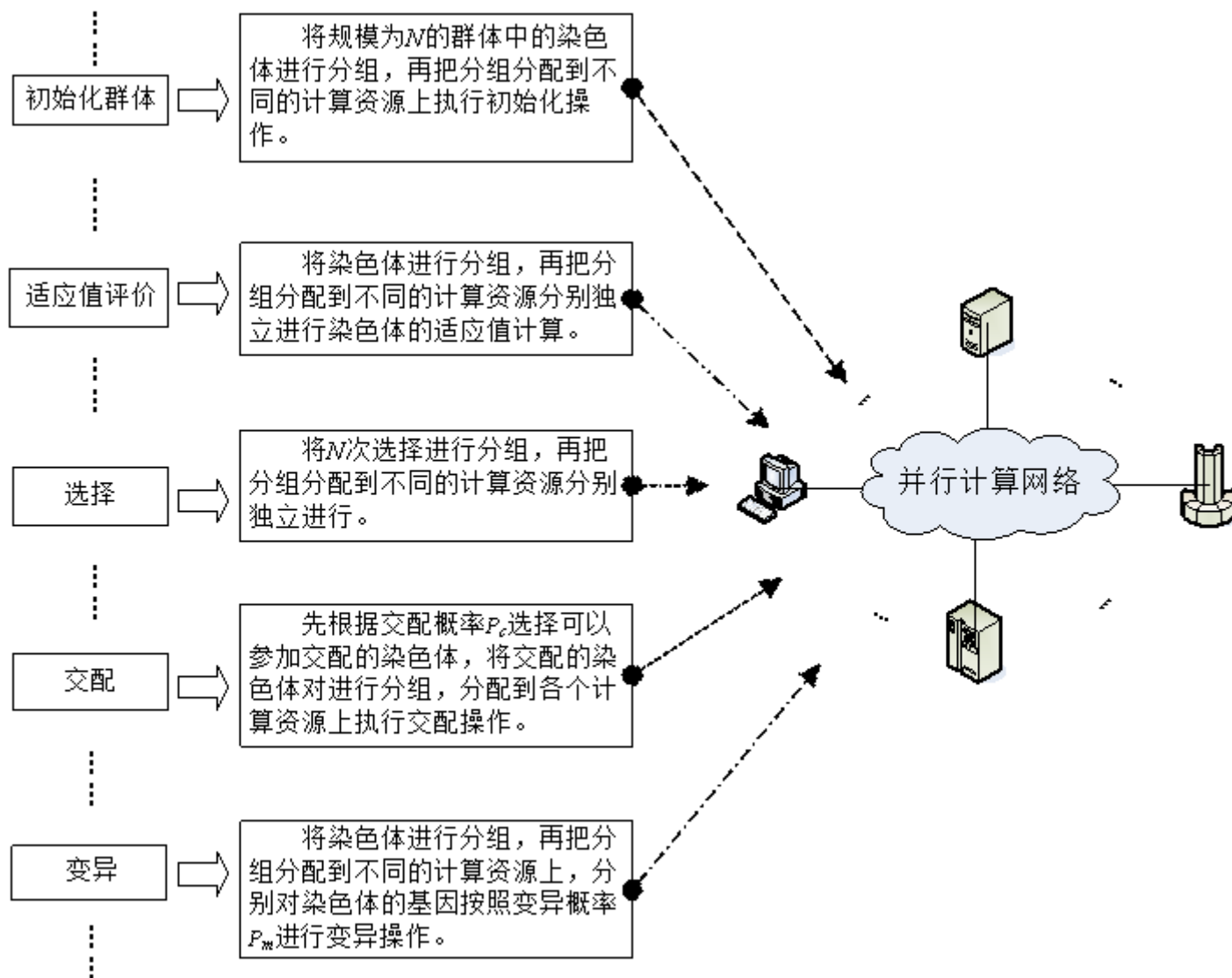
- 单指令流多数据流计算机
- 多指令流多数据流计算机
- 并行计算网络

- 串行计算

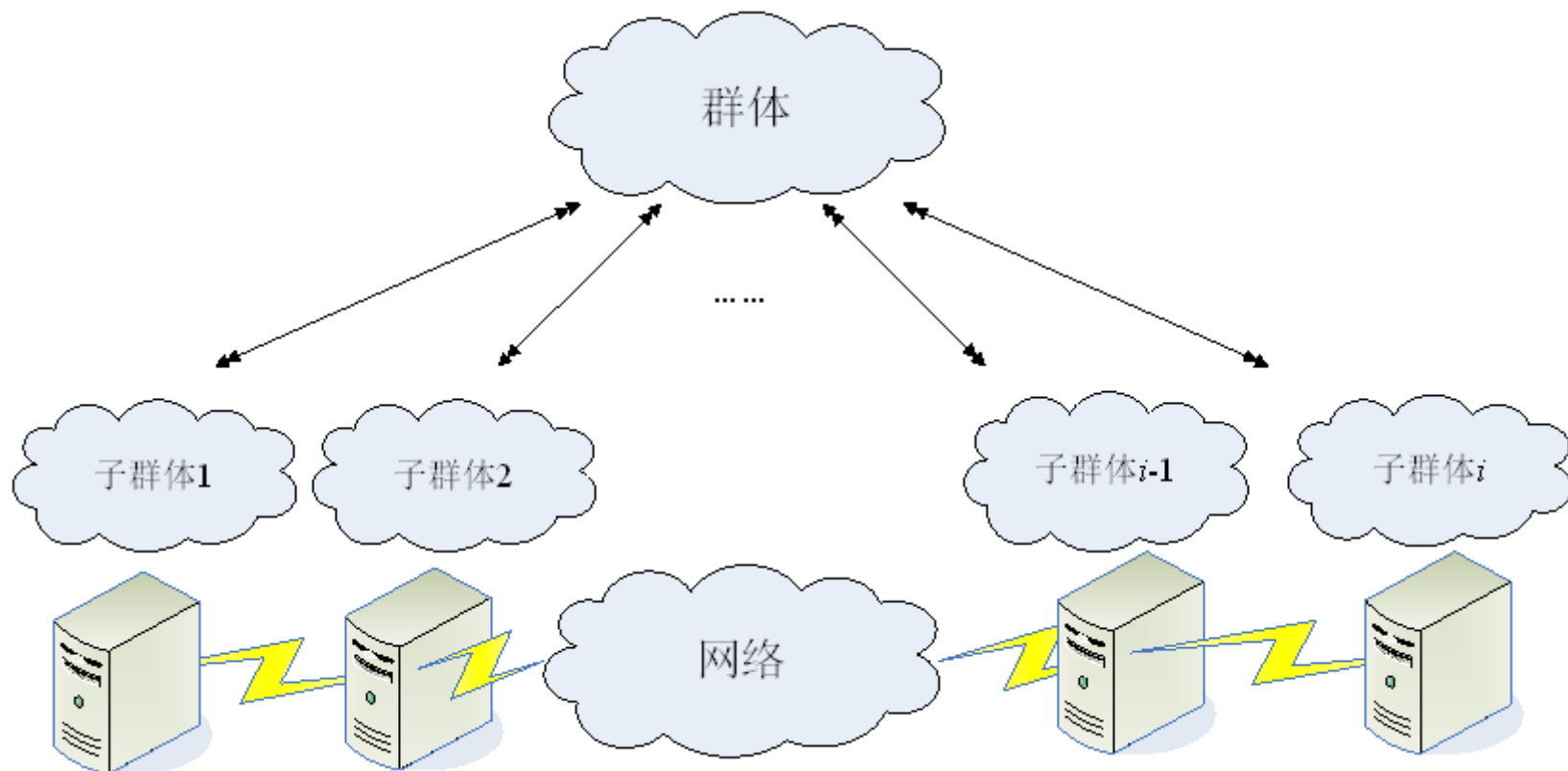
- 单指令流单数据流处理器



标准型并行方法



分解型并行方法



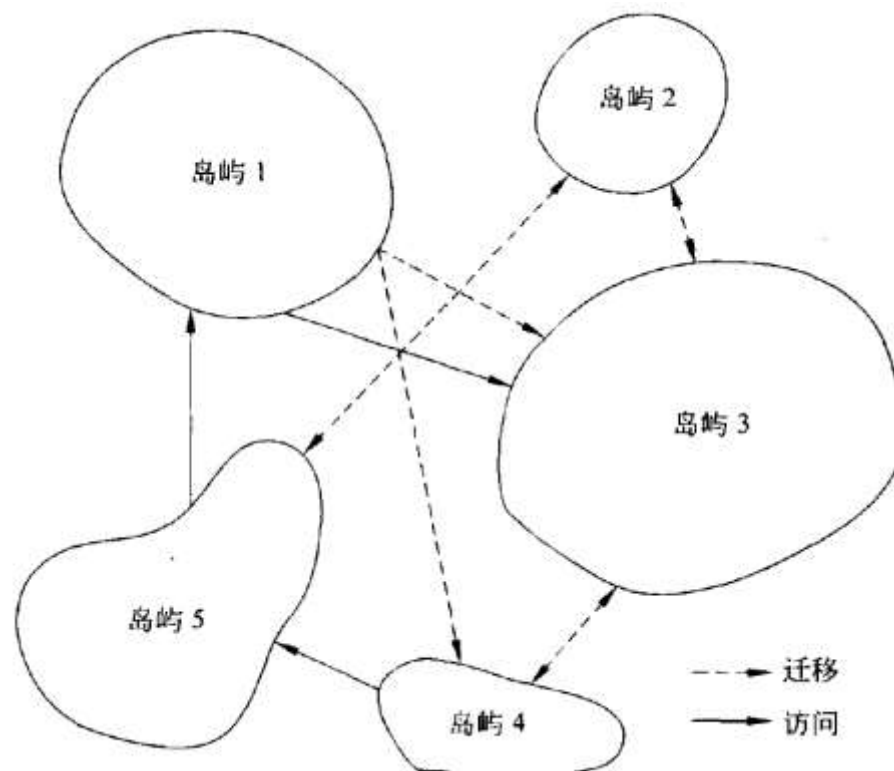
分解型并行方法

子群体进化信息交换问题

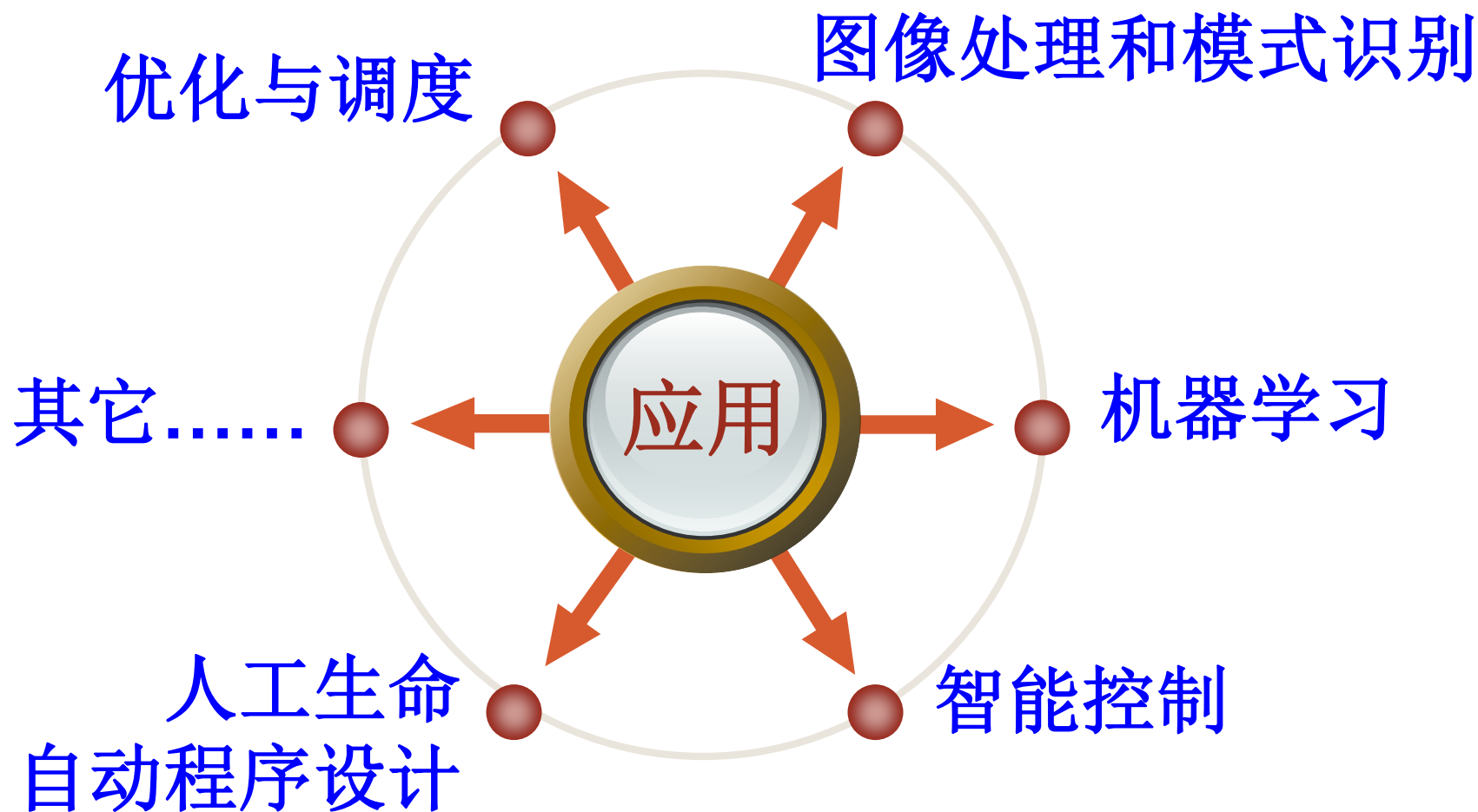
- ◆ 交换的时间
- ◆ 交换的方式
- ◆ 交换的内容

群体模型

- ◆ 岛屿模型
- ◆ 踏脚石模型
- ◆ 邻居模型



遗传算法的应用



2018年诺贝尔化学奖

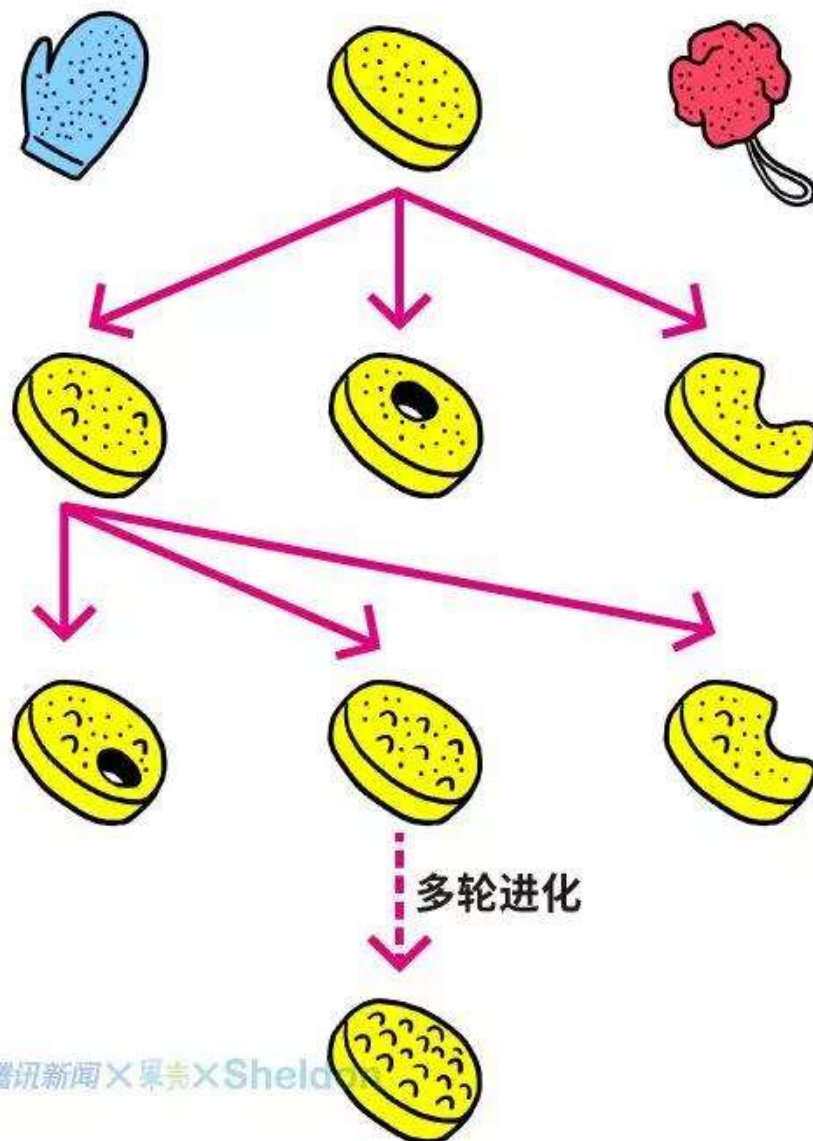
- 2018年10月3日，美国化学家弗朗西斯·H·阿诺德、美国生物化学家乔治·P·史密斯（George P. Smith）和英国生物化学家格雷戈里·P·温特爵士、荣获2018年诺贝尔化学奖。
- “控制”了进化，由人为定向进化产生的酶，已经被用于制造包括生物燃料到药物的各种物质。而通过一种名叫“噬菌体展示”（phage display）的方法产生的抗体，可以用于对抗人类的自身免疫疾病，并且在一些情况下治愈转移性癌症。

你可以把蛋白酶理解成搓澡巾



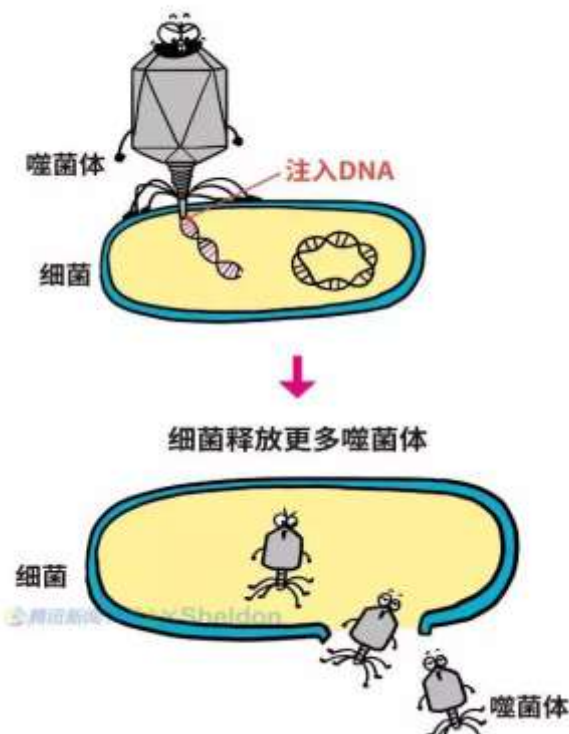
回到第一步, 让好用的搓澡巾基因
产生进一步变异, 并反复循环。

搓澡巾(蛋白酶)的定向进化



最好的搓澡巾(蛋白酶)

噬菌体感染细菌的过程



第1次循环



第2次循环



第3次循环

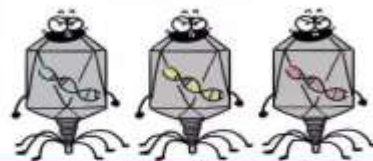
史密斯发明了噬菌体呈递技术，我用这种技术来生产抗体！



温特

你可以把医用抗体理解成钥匙

第一步：
向噬菌体的钥匙基因
引入随机的变异



第二步：
变异后的基因产生各种
不同的钥匙



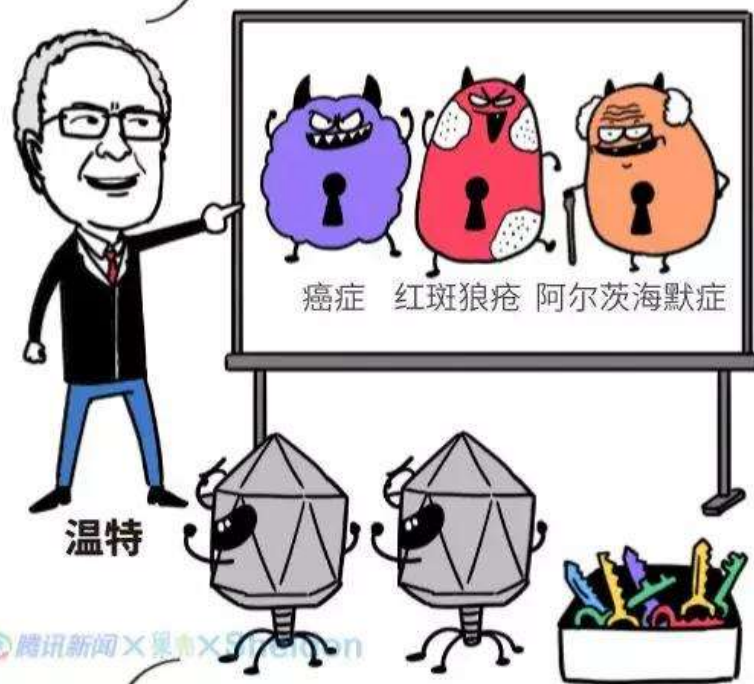
第三步：
用预制的锁测试钥匙，谁的钥匙更
匹配就留下谁的钥匙基因



X 3

回到第一步，让好用的钥匙基因
产生进一步变异，并循环3次

你们给我搞定这三种病！



腾讯新闻 X 果壳 X 51cto

包在我身上了！

配钥匙我最在行了！

THE NOBEL PRIZE IN CHEMISTRY 2018

Illustrations: Niklas Elmehed



Frances H.
Arnold

“for the directed
evolution
of enzymes”

George P.
Smith

“for the phage display of peptides
and antibodies”

Sir Gregory P.
Winter

THE ROYAL SWEDISH ACADEMY OF SCIENCES

总结

- 优化算法：
 - 解析法：只能得到局部最优解，且要求目标函数连续光滑及可微信息；
 - 枚举法：虽然克服了这些缺点，但计算效率太低，且对于实际问题往往由于搜索空间大而不能将所有情况都搜索到；
 - 动态规划法：存在“指数爆炸”问题，它对于中等规模和适度复杂性的问题，也常常无能为力。

■ 同以上算法相比，遗传算法有以下特点：

- ① 遗传算法是对决策变量的编码进行操作，这样提供的参数信息量大，优化效果好。
- ② 遗传算法是从许多点开始并行操作，因而可以有效地防止搜索过程收敛于局部最优解。算法具有并行计算的特点，可通过大规模并行计算来提高计算速度。
- ③ 遗传算法通过目标函数来计算适配值，而不需要其他推导和附加信息，对问题的依赖性小。对于待寻优的函数基本无限制，应用范围较广。

- ④ 遗传算法的寻优规则是由概率决定的，而非确定性的。
- ⑤ 遗传算法在解空间进行高效启发式搜索，而非盲目地穷举或完全随机搜索。
- ⑥ 遗传算法计算简单，功能强，更适合大规模复杂问题的优化。